DX100: A Programmable Data Access Accelerator for Indirection

Alireza Khadem[†*], Kamalavasan Kamalakkannan[‡*], Zhenyan Zhu[†], Akash Poptani[†], Yufeng Gu[†], Jered Benjamin Dominguez-Trujillo[‡], Nishil Talati[†], Daichi Fujiki, Scott Mahlke[†], Galen Shipman[‡], Reetuparna Das[†]
[†]University of Michigan, [‡]Los Alamos National Laboratory
{arkhadem,reetudas}@umich.edu

arXiv:2505.23073v1 [cs.AR] 29 May 2025

Abstract

Indirect memory accesses frequently appear in applications where memory bandwidth is a critical bottleneck. Prior indirect memory access proposals, such as indirect prefetchers, runahead execution, fetchers, and decoupled access/execute architectures, primarily focus on improving memory access latency by loading data ahead of computation but still rely on the DRAM controllers to reorder memory requests and enhance memory bandwidth utilization. DRAM controllers have limited visibility to future memory accesses due to the small capacity of request buffers and the restricted memory-level parallelism of conventional core and memory systems.

We introduce DX100, a programmable data access accelerator for indirect memory accesses. DX100 is shared across cores to offload bulk indirect memory accesses and associated address calculation operations. DX100 *reorders*, *interleaves*, and *coalesces* memory requests to improve DRAM row-buffer hit rate and memory bandwidth utilization. DX100 provides a general-purpose ISA to support diverse access types, loop patterns, conditional accesses, and address calculations. To support this accelerator without significant programming efforts, we discuss a set of MLIR compiler passes that automatically transform legacy code to utilize DX100. Experimental evaluations on 12 benchmarks spanning scientific computing, database, and graph applications show that DX100 achieves performance improvements of 2.6× over a multicore baseline and 2.0× over the state-of-the-art indirect prefetcher.

1 Introduction

Scientific and engineering simulations are critical tools used in nuclear security, materials research, energy production, aerospace, and the basic sciences [113]. Many of these simulations routinely require weeks or even months to complete on the largest-scale supercomputers in the world, primarily bottlenecked by *data access* with very low arithmetic intensity. For instance, state-of-the-art Adaptive Mesh Refinement (AMR) simulations [20, 34] demand ≈2 PB of memory and heavily rely on sparse data structures, with over 50% of instructions involving *indirect memory accesses* [112]. In fact, the first goal of DoE's next-generation supercomputer, ATS-5, to be commissioned in 2027, is "Overcoming the memory wall: continued memory bandwidth performance improvements for tri-lab applications" [70].

Beyond high-performance computing, indirect memory accesses, where memory address depends on another load (*e.g.*, `A[B[i]]`), are common in several data-intensive workloads. These accesses are employed in sparse linear algebra [30, 66] and machine learning applications [38] to use sparse data structures such as CSR and CSC, graph analytics [15, 22, 74, 111, 128] to traverse graph nodes and edges, and in-memory database applications [11, 12] to join tables using hash-join algorithms.

Memory bandwidth is crucial for the performance of these applications, as indirect accesses often miss caches due to low spatial and temporal locality. Additionally, these accesses exhibit poor memory bandwidth utilization on multi-core systems for several reasons: **First,** subsequent indirect loads often *access non-contiguous memory locations* in different DRAM rows. Further, concurrent memory requests from multiple cores to different rows of the same bank lead to row conflicts. Therefore, the DRAM row-buffer hit rate drops significantly. **Second,** *memory-level parallelism (MLP) is constrained* by structural limitations within the core and memory system [13], such as ROB, LSQ, and cache MSHRs. Additionally, indirect accesses depend on prior memory accesses and address calculation instructions, creating a chain of dependencies and limiting outstanding loads and stores in the core [14]. As a result, DRAM controllers have limited visibility to future accesses, hindering their ability to optimize the order of DRAM commands and improve bandwidth utilization [47, 49, 73, 85, 102, 103, 108]. Previous approaches such as runahead execution [29, 39, 84, 86] enhance the memory access latency by improving cache hit rate but remain bounded by core's structural limitations.

The research community has focused extensively on compute acceleration [24–26, 32, 36, 38, 40, 69, 95, 96, 99, 101, 134], but less attention has been paid to building accelerator architectures for data access [65, 123, 130]. Data-centric accelerator solutions can enrich future computing systems especially when memory access and data movement costs dominate [45]. **This work proposes DX100, a programmable data access accelerator that enables the offloading of *bulk* indirect loads, stores, and read-modify-write (RMW) operations.** Accelerating bulk memory accesses using DX100 significantly reduces the core's instruction footprint. More importantly, to enhance the memory bandwidth utilization, DX100 is inspired by the following key ideas: (a) Offloading indirect memory accesses and address calculation instructions to an accelerator near the memory controllers improves the effective memory-level parallelism. (b) Indirect address issue to DRAM can

*Alireza Khadem and Kamalavasan Kamalakkannan are co-first authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
ISCA '25, Tokyo, Japan
This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/2025/06
https://doi.org/10.1145/3695053.3731015

be optimized for *bulk* memory operations by leveraging the visibility of a large window of indices. Consider a bulk access `A[B[i]]` for `i` ranging from 0 to 16K. DX100 has the visibility of all 16K indices after fetching them. So, DX100 *reorders* them to improve the row-buffer hit rate, *coalesces* them to reduce accesses, and *interleaves* them to enhance DRAM channel and bank-group interleaving.

DX100 provides a general-purpose ISA with eight instructions to support a diverse set of loop patterns (*single* and *range*), loop condition patterns, multiple levels of indirection, address calculation operations (*scalar* and *vector*), and access types (*load*, *store*, and *RMW*). To enable these instructions, DX100 integrates four functional units: (a) *Stream* unit handles streaming loads and stores, (b) *Indirect* unit manages indirect loads, stores, and RMWs while facilitating reordering, coalescing, and interleaving for improving memory bandwidth utilization, (c) *Range Fuser* unit supports merging range loops to enable bulk access, and (d) *ALU* unit performs comparison and arithmetic operations. A scratchpad unit stores the intermediate data and facilitates the communication between DX100's functional units and the CPU cores. While DX100 provides a flexible ISA, re-developing legacy code with these APIs could be cumbersome. Therefore, we develop a compiler infrastructure using the MLIR framework [63] and Polygeist [77] to automatically detect arbitrary indirect memory accesses within legacy code, hoist them, and transform them into DX100 instructions.

Similar to DX100, Fetcher units [65, 130] and Decoupled Access Execute (DAE) architectures [115] hoist memory access instruction streams and offload them to tightly-coupled access coprocessors, improving the core structural limitations. Distinct from these works, DX100 is shared among cores and directly accesses the main memory, eliminating the bottleneck of core and memory system structures. More importantly, these solutions remain bounded by the random memory access patterns as they do not *reorder* and *interleave* memory accesses across a tile of bulk indices. Consequently, they provide modest improvements in memory bandwidth utilization primarily by increasing the memory access rate.

In summary, this paper offers the following contributions:

- We propose DX100, a programmable data access accelerator that supports offloading indirect and streaming memory accesses. DX100 enhances memory bandwidth utilization by optimizing *bulk* accesses: it *reorders* to improve DRAM row-buffer hits, *coalesces* to reduce accesses, and *interleaves* to enhance DRAM channel and bank-group parallelism.
- The proposed architecture and ISA are designed for integration with arbitrary applications that can benefit from bulk memory accesses. DX100 is flexible to support multiple degrees of indirection, diverse sets of loop patterns, loop conditions, data access types (load, store, RMW), and various address calculation operations.
- We accurately evaluate DX100 as a memory-mapped accelerator using execution-based and event-driven simulators [21, 71]. We study several benchmark suites, such as NAS [7], GAP [17], Hash-Join [11], UME [42], and Spatter [64]. DX100 achieves a 2.6× performance improvement over the multicore baseline and outperforms the state-of-the-art hardware indirect prefetcher, DMP [33], by 2.0×.

2 Background and Motivation

2.1 DRAM Organization and Access Reordering

Memory access patterns play a crucial role in bandwidth utilization due to the organization and timing constraints of DRAM. Figure 1 (a) illustrates the DRAM organization of DDR4 technology. Memory addresses are distributed across multiple channels, ranks, four bank groups, and four banks. Each bank consists of 2D DRAM arrays arranged in rows and columns of DRAM cells. This hierarchical organization allows channels, ranks, bank groups, and banks to process different DRAM requests independently. Such parallelism is essential for fully utilizing DRAM bandwidth.

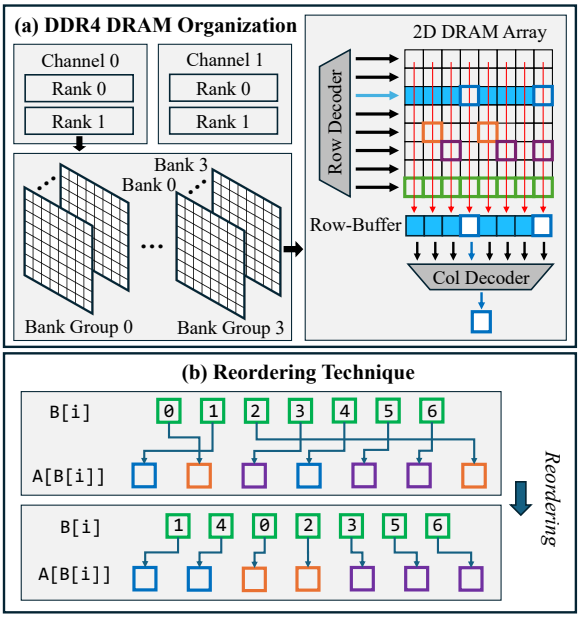


Figure 1: (a) DDR4 DRAM organization, (b) memory access reordering technique for improving row buffer hit rate.

Accessing data in a DRAM bank requires opening the corresponding row. This is done with a precharge (`PRE`) command to close the currently open row and then an activate (`ACT`) command to open the desired row. Once open, the data is loaded into the Row Buffer, which enables fast column accesses to the same row. Indirect memory accesses often span multiple rows, missing the row buffer and requiring frequent row switchings. Row buffer misses substantially reduce DRAM bandwidth utilization due to the long timing constraints imposed between the `PRE` and `ACT` commands.

To improve the row buffer hit rate, memory controllers often *reorder* the outstanding memory accesses as shown in Figure 1 (b). Both accesses targeting the blue row are issued first (A[B[1], A[B[4]). Typically, this reordering is constrained to a window of 32 to 128 accesses to balance hardware cost and complexity. This window size is not enough for applications with indirect memory access and higher sparsity, leading to lower bandwidth utilization. DX100 architecture and programming model provide memory-bound applications the opportunity to offload bulk memory accesses to an accelerator, *increasing the reordering window* to up to 16K accesses.

Even with ideal row buffer hit rates, interleaving accesses across channels and bank groups is essential for optimal DRAM bandwidth

utilization. Channel interleaving is crucial because channels operate independently and can handle memory requests at a high rate. Furthermore, consecutive accesses to the same bank group reduce bandwidth utilization due to a long column-to-column timing constraint t_{CCDL}. To fully exploit the bandwidth, subsequent memory requests should be interleaved across different bank groups, reducing the effective column-to-column timing constraint to t_{CCDS}, which is half of t_{CCDL}. DX100 also interleaves the accesses to different channels and bank groups, effectively optimizing the utilized channel bandwidth.

2.2 Other Challenges of Indirect Accesses

Another key challenge for high memory bandwidth utilization is the memory request rate. For instance, under the ideal assumptions of a 100% cache hit rate, 100% DRAM row-buffer hit rate, and perfect interleaving, fully utilizing our evaluated system with 204.8 GB/s Last Level Cache (LLC) bandwidth and 51.2 GB/s DDR4 memory bandwidth requires one LLC request per cycle and one DRAM request every four cycles. However, conventional microarchitectures struggle to reach this rate due to several limiting factors.

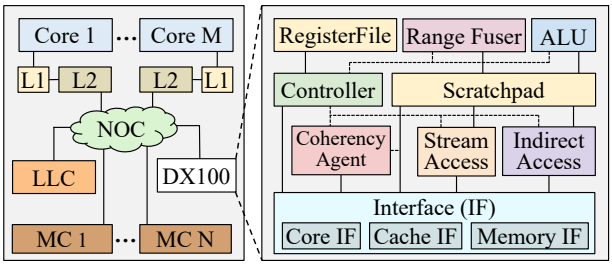
First, indirect memory accesses rely on prior index loads and address calculations, forming a *chain of instruction dependencies* that restricts the number of outstanding memory accesses [14]. DX100 breaks this dependency chain by hoisting or sinking bulk memory accesses before or after the loop and issuing bulk accesses independently. Moreover, address calculations add substantial overhead to the *dynamic instruction count*, resulting in significant execution time and energy consumption [112]. DX100 addresses this by employing custom address calculation components to accelerate bulk accesses and significantly reduce core instruction count.

Second, conventional microarchitecture contains a *hierarchy of buffers* that further restricts memory-level parallelism, such as the Reorder Buffer and Load/Store Queues in the processor core, as well as the Miss Status Holding Registers (MSHRs) in the cache. DX100 eliminates this bottleneck by bypassing intermediate buffers and directly injecting DRAM requests into the memory controller, while taking the coherency protocol into consideration.

Finally, multi-threaded execution of Read-Modify-Write (RMW) operations on conventional multi-core systems requires *fine-grained atomicity*, which further constrains memory-level parallelism. Atomic RMW operations use memory fences that serialize memory requests [4] and rely on cacheline or bus locking mechanisms [48]. In contrast, DX100 is shared across a group of cores and maintains exclusive write access to indirect memory regions, eliminating the need for fine-grained atomic operations.

3 DX100 Architecture

This section delves into the details of DX100 microarchitecture. We provide an overview first, before describing individual functional units and system interfaces for the accelerator. DX100 executes hoisted bulk memory accesses at a granularity of **tile** (*e.g.,* 16K 4B words). The CPU cores perform all remaining computations. Figure 2 (a) illustrates the placement of the DX100 as a shared accelerator within the processor. DX100 is integrated as a modular memory-mapped component within the coherent fabric connected via the network-on-chip to system components, requiring minimal



(a) System Integration (b) DX100 Architecture

Figure 2: (a) DX100 is integrated as a shared memory-mapped accelerator to the system. (b) DX100 architecture.

adjustments to the core, memory controller, or instruction-set architecture. DX100 is designed as an independent component for flexible integration with mesh [59, 120] or ring [121] interconnects.

DX100 is shared among a group of cores within the processor. While we focus on a single DX100 instance throughout the paper, we explore the scalability of multiple instances in Section 6.6. Compared to previous fetcher units that use private co-processors, the shared design offers several key benefits: *First*, the area overhead of a shared accelerator is amortized across multiple cores. A memory-mapped accelerator requires minimal modifications to the core, as opposed to the co-processors, which introduce new instructions to the ISA or modify the LSQ. *Second*, a shared accelerator can reorder memory accesses across threads from different cores in a shared-memory environment (*e.g.,* Pthread or OpenMP), further improving row buffer hit rates and enabling higher coalescing factors. *Third*, DX100 does not require fine-grain atomic accesses for store and RMW operations as it maintains exclusive access to the indirect memory regions. Considering these advantages, we designed DX100 as a shared accelerator. The primary drawback of this approach is slower communication between cores and the accelerator. To mitigate this, we make the core's *data* accesses to DX100 (*e.g.,* for loading gathered data) cacheable, enabling stride prefetchers to effectively reduce the data access latency (Section 3.6).

Figure 2 (b) provides a high-level overview of the DX100 architecture. *Scratchpad* contains multiple tiles that are the source and destination of DX100 instructions, facilitating communication between functional units and between DX100 and cores. All functional units within DX100 operate in `TILE` granularity (*e.g.,* 16K 4B words). *Stream Access* performs streaming loads and stores (*e.g.,* `B[i]`) from the LLC to the Scratchpad. *Indirect Access* reads index values from the Scratchpad and performs indirect load, store, or RMW (*e.g.,* `A[B[i]]`) from LLC or memory. *Range Fuser* combines multiple small range loops with few iterations into a single larger loop for efficient bulk access with a large tile size. *ALU Unit* performs condition calculations and index value transformations. *Controller* manages the dispatch, issue, and retirement of DX100 instructions. *Coherency Agent* tracks cached scratchpad data to ensure coherence. *Interface* manages DX100 accesses to the cache and memory, as well as core accesses to DX100. Finally, *Register File* holds scalar values required for loops and ALU operations.

Table 1: Common Data Access Patterns of Irregular Applications

Benchmark	Kernel	Access	Condition/Address Calculation	Loop
NAS [7]	IS	RMW A[B[i]]	–	i = F to G
	CG	LD A[B[j]]	–	j = H[i] to H[i+1]
GAP [17]	BFS	ST A[B[j]]	if (D[E[j]] < F)	j = H[K[i]] to H[K[i]+1]
	BC	RMW A[B[j]]	if (D[E[j]] == F)	j = H[K[i]] to H[K[i]+1]
	PR	RMW A[B[j]]	–	j = H[i] to H[i+1]
Hash-Join [11]	PRH	ST A[B[f(C[i])]]	f(C[i]) = (C[i] & F) >> G	i = F to G
	PRO	ST A[B[f(C[i])]]	f(C[i]) = (C[i] & F) >> G	i = F to G
UME [42]	GZZ	RMW A[B[[i]]]	if (D[i] >= F)	i = F to G
	GZZI	LD A[B[C[j]]]	if (D[j] >= F)	j = H[K[i]] to H[K[i]+1]
	GZP	RMW A[B[i]]	if (D[i] >= F)	i = F to G
	GZPI	LD A[B[C[j]]]	if (D[j] >= F)	j = H[K[i]] to H[K[i]+1]
Spatter [64]	XRAGE	ST A[B[i]]	–	i = F to G

*We show only one pattern for each kernel because of space limitation.

Table 2: DX100 Instruction Set

Type	Opcode	Operands
Indirect Access	ILD	DTYPE BASE TD TS1 TC
	IST	DTYPE BASE TS1 TS2 TC
	IRMW	DTYPE BASE OP TS1 TS2 TC
Stream Access	SLD	DTYPE BASE TD RS1 RS2 RS3 TC
	SST	DTYPE BASE TS RS1 RS2 RS3 TC
ALU	ALUV	DTYPE OP TD TS1 TS2 TC
	ALUS	DTYPE OP TD TS RS TC
Range Loop	RNG	TD1 TD2 TS1 TS2 RS1 TC

3.1 Common Patterns and ISA

Table 1 demonstrates access, condition, address calculation, and loop patterns in evaluated irregular workloads with the following key takeaways: *First*, in addition to indirect loads (`C = A[B[i]]`), store (`A[B[i]] = C`) and read-modify-write (RMW) operations (`A[B[i]] += C`) are frequently used to update values at indirect addresses. *Second*, supporting stores and RMWs is challenging because they are often conditioned. Unlike loads, where ignoring conditions may only result in performance degradation by fetching extra data, neglecting conditions for store and RMW compromises the correctness. *Third*, besides single loops (`i = F to G`), range loops are commonly employed in two forms: direct range (`j = H[i] to H[i+1]`) where the loop induction variable `j` increments sequentially, or indirect range (`j = H[K[i]] to H[K[i]+1]`) where the ranges themselves depend on indirect accesses. *Fourth*, indirect range loops usually cover only a few iterations. A common example is graph workloads with frontier queues, where a range loop iterates over a few neighbors of the current frontier nodes. *Finally*, multiple levels of indirection (`A[B[C[i]]]`) are common.

DX100 ISA seamlessly supports these patterns with eight instructions as shown in Table 2. The ISA (1) *Decouples* streaming accesses from indirect accesses. Streaming accesses use scratchpad tiles (`TD`) as the destination. Indirect accesses use scratchpad tiles as index source (`TS`). (2) Supports different access types (`LD, ST, RMW`). (3) Allows various `ALU` operations (`OP` operand) for conditional memory accesses using the condition tile (`TC`). (4) Enables fusing range loops (`RNG`) for bulk accesses. (5) Supports various data types (`DTYPE`), such as *u32*, *i32*, *f32*, *u64*, *i64*, and *f64*, and different operations (`OP`), such as *ADD*, *SUB*, *MUL*, *MIN*, *MAX*, *AND*, *OR*, *XOR*, *SHR*, *SHL*, *LT*, *LE*, *GT*, *GE*, and *EQ* required for a general-purpose usage. Note, DX100 only supports a subset of associative and commutative operations, such as *ADD*, *MAX*, and *MIN* for the *IRMW* instructions as they reorder the operations. These features, using *scratchpad as an intermediate storage*, enable offloading conditional loops, multiple levels of indirection, and complex address calculations.

3.2 Indirect Access Unit

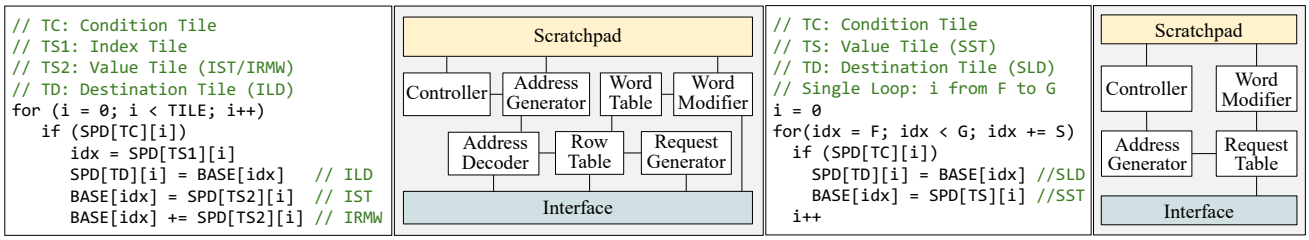
Functionality: Figure 3 (a) shows the functionality of the Indirect Access unit, which performs load, store, and RMW operations on indirect memory locations. The index tile (`TS1`) has been fetched and exists in the scratchpad. `TS1` can originate from streaming access for single-level indirection (`A[B[i]]`), indirect access for multi-level indirection (`A[B[C[i]]]`), or the ALU unit for complex address calculations (`A[f(C[i])]`).

Memory Bandwidth Enhancements: Figure 3 (b) details the architecture of this unit, which is designed to optimize DRAM bandwidth utilization. The Indirect Access unit employs a Row Table to *reorder* indirect addresses by storing addresses belonging to the same DRAM row together, and issuing them at the same time. This approach ensures that all accesses to columns within the same DRAM row are completed consecutively, increasing the row buffer hit rate. Additionally, a Word Table organizes target words within each column into a linked list. This component enables the unit to access unique columns, effectively *coalescing* redundant accesses within a tile. Lastly, Request Generator *interleaves* accesses across different DRAM channels and bank groups, maximizing channel bandwidth and enabling bank-group interleaving.

Row Table Architecture. Figure 4 (a) shows the Row Table, which consists of multiple slices corresponding to the DRAM banks. This design enables high-throughput lookups for row and column addresses. Figure 4 (b) shows the slice architecture, which stores row and column information for outstanding requests to a specific DRAM bank. Each slice includes a Binary Content Addressable Memory (BCAM) and an SRAM cell. BCAM offers fully associative lookup access to the information of 64 target DRAM rows. It stores a valid bit (`V`), a sent bit (`S`) indicating whether all requests to the row have been issued, and the row address (`RO`). SRAM cell holds the information of up to 8 DRAM columns per row, such as a valid bit (`V`), a sent bit (`S`), a cache hit bit (`H`) indicating if the column is present in the caches, the column address (`CO`), and a pointer to the Word Table (`Tail i`) targeting words within the column.

Word Table Architecture. Figure 4 (c) depicts the Word Table, which holds the word information in target DRAM columns using a linked-list structure. For each word in the tile corresponding to iteration number `i`, Word Table stores a valid bit (`V`), the word's offset within the column (`WO`), and the previous iteration (`Previous i`) accessing the same column, enabling the linked-list organization.

The Indirect Access unit operates in three stages: **Operation Stage 1 - Fill:** Controller reads the condition values (`SPD[TC][i]`) from the Scratchpad. If condition holds, it triggers Address Generator with the iteration number (`i`). Address Generator fetches the index value (`idx=SPD[TS1][i]`) from the Scratchpad and calculates the virtual word address (`BASE[idx]`). Address Decoder translates the virtual address to the physical address using Interface. Then, it maps the physical address to DRAM coordinates (`CH`, `RA`, `BG`, `BA`, `RO`, and `CO`) and word offset (`WO`), which are inserted into Row and Word Tables in 4 steps: (a) The Row Table slice is determined based on the `CH`, `RA`, `BG`, and `BA` target addresses. (b) The BCAM cell of the



(a) Indirect Unit Functionality (b) Indirect Unit Architecture (c) Stream Unit Functionality (d) Stream Unit Architecture

Figure 3: Stream and Indirect Access unit functionality and architecture.

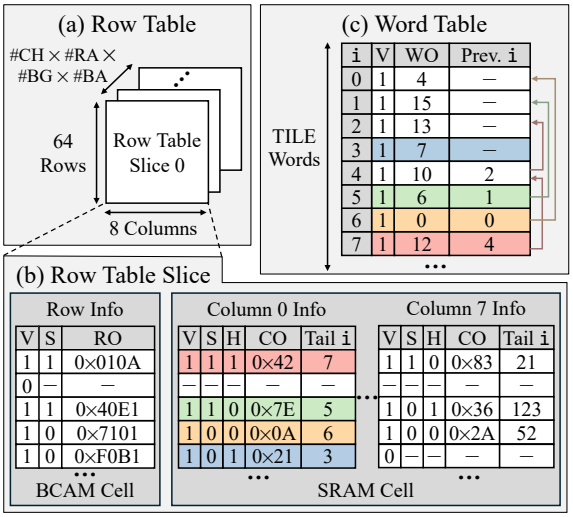


Figure 4: (a) Row Table, (b) Row Table Slice, and (c) Word Table architecture of Indirect Access unit.

identified slice looks for a valid and unsent entry with the target row address `RO`. Once the row is selected, the SRAM cell finds a valid and unsent entry with the target column address `CO`, captures its `Tail i` field, and updates it to the iteration number of the current target element. If no such entry exists in the BCAM or SRAM cells, the Row Table allocates a new entry. When a DRAM column is accessed for the first time, the Row Table queries the Interface to determine the cache hit status (`H` bit, see Section 3.6). (c) The new word offset `WO` is inserted into the ith entry of the Word Table. The `Previous i` field of this entry is set with `Tail i` from Row Table, linking the current word to the prior words in the same column.

Operation Stage 2 - Request: Once all words are inserted for a row or the Row Table reaches capacity, the Indirect Access unit initiates the process of generating and issuing memory accesses to the Interface. For that, each Row Table slice sequentially scans its BCAM and SRAM cells to identify the valid and unsent `RO` and `CO` addresses, as well as the cache hit bit (`H`). The Request Generator arbitrates among Row Table slices in a predetermined order, interleaving slices corresponding to different DRAM channels and bank groups for optimized bandwidth utilization. Using `RO` and `CO` addresses, and the slice's bank coordinates, the Request Generator calculates the target memory address. This information, along with the `H` bit, is then sent to the Interface to load the required data.

Operation Stage 3 - Response: When a cache line is received from the Interface, the Address Decoder maps its address to DRAM coordinates. The Row Table looks for valid and sent columns in its corresponding slice to identify the tail of the word linked-list (`Tail i` field). The Word Table traverses this linked-list to retrieve the word offsets (`WO`) and iteration numbers (`i`). Word Modifier then processes the data based on the instruction type: *Indirect Load* (`ILD`): It extracts the required words from the response data and writes them to the destination Scratchpad tile (`TD`). *Indirect Store* (`IST`): It reads the new words from the source tile (`TS2`), and inserts them into the response data. *Indirect RMW* (`IRMW`): It reads new words from the source tile, extracts words from the response data, performs arithmetic operations, and inserts the modified words back into the response data. For store and RMW operations, modified data is sent back to the Interface to complete the write operation.

3.3 Stream Access Unit

Figure 3 (c) shows the functionality of the Stream Access Unit. The streaming load (`SLD`) instruction loads a tile of data elements from sequential addresses in a single loop and writes them to the scratchpad. The streaming store (`SST`) instruction stores a tile of values from the scratchpad to sequential memory addresses. Figure 3 (d) demonstrates the architecture of this unit. Controller generates loop iteration values (`idx` and `i`). In conditioned instructions, Controller retrieves condition values from the scratchpad and skips iterations where the condition is not met. Address Generator computes the streaming virtual cache line addresses and word offsets (`wid`) within each cache line. Request Table functions similarly to MSHR by tracking outstanding memory addresses and their `wid` and `i` values. It performs the address translation using the Interface, generates load requests, and injects them back into the Interface. Upon receiving responses, Request Table retrieves the associated `wid` and `i` values. For `SLD`, Word Modifier extracts specific words from the cache line and writes them to the scratchpad. For `SST`, Word Modifier reads the new values from the scratchpad, inserts them to the cache line, and issues a store request to the Interface.

Finally, note that the cores can efficiently handle streaming accesses (`B[i]`) and some address calculation operations. Performing these tasks on the core and then transferring the data to DX100 would simplify the accelerator design, allowing it to focus specifically on indirect accesses with lower memory bandwidth utilization. However, this approach would increase the data transfer between the core and DX100, making the DX100 interface a bottleneck. As a result, the accelerator would frequently stall while waiting for the required data for indirect access. This reduces the rate of DX100 memory requests and the memory bandwidth utilization. Therefore, we decided to support all these patterns locally within DX100.

3.4 Range Fuser and ALU Units

Section 3.1 showed that range loops like `j = H[K[i]] to H[K[i]+1]` involve few iterations, which is inefficient for offloading bulk accesses to DX100. To address this limitation, we provision a **Range Fuser** unit, which combines multiple small-range loops into a single larger range. Figure 5 illustrates the functionality of the Range Fuser unit. It processes two input tiles representing the minimum and maximum range boundaries (`H[K[i]]` and `H[K[i]+1]`). These boundaries can be loaded either by the Stream Access unit for direct range loops or by the Indirect Access unit for indirect range loops. The Range Fuser generates two output tiles that contain the induction variable values of the outer single loop (`i`) and the inner range loop (`j`). These tiles can then be passed to the Indirect Access unit to facilitate further memory accesses, such as `A[B[j]]`.

```
// TC: Tile Condition
// TS1/TS2: Range Boundaries
// TD1/TD2: i and j loop iteration values
idx = 0
for (i = 0; i < TILE; i++)
   if (SPD[TC][i])
      for (j = SPD[TS1][i]; j < SPD[TS2][i]; j++)
         SPD[TD1][idx] = i
         SPD[TD2][idx] = j
         idx++
```

Figure 5: Range Fuser Functionality.

The **ALU** unit executes arithmetic, bitwise, and comparison operations required for condition evaluation and address calculations (refer to Table 1). The ALU operands can be two tiles (`D[i] > E[i]`) or a tile and a scalar value from the register file (`C[i] & F`). The output condition tiles can be utilized by all functional units for subsequent operations, and address calculation results can be used as indices for indirect memory accesses.

3.5 Controller, Register File, and Scratchpad

Controller receives DX100 instructions from the cores and schedules them through multiple stages: dispatch, issue, execute, and retire. Each DX100 instruction is 192b wide and is transmitted via three 64b memory-mapped stores. To support out-of-order execution, Controller employs a scoreboard. To avoid WAW and RAW hazards without register renaming, Controller prevents the dispatch of instructions targeting destination tiles that are already in use by a valid instruction in the scoreboard.

Register File contains scalar values, which are used for single-loop boundaries, loop strides, and ALU operations. **Scratchpad** stores multiple tiles, each containing `TILE` elements. For each tile, Scratchpad maintains a size and a ready bit. When an instruction is dispatched, ready bits corresponding to all source and destination tiles are set to 0, indicating that the tiles have not finished the execution yet. Once the instruction retires, these bits are reset to 1. This bit allows the synchronization between cores with DX100.

In addition to ready bits, Scratchpad maintains a finish bit for all elements. When an instruction is issued to a functional unit, the finish bits of all destination tile elements are set to 0. As the elements are computed and written back to the Scratchpad, their finish bits are set to 1. This design enables fine-grained execution coordination between functional units. For example, as soon as an element of `B[i]` is loaded into the Scratchpad by the Stream Access unit, the Indirect Access unit can immediately access it and populate the Row Table and Word Table for memory operations like `A[B[i]]`. This overlapping of operations hides the fill latency of the Indirect Access unit behind the load of the index array.

3.6 Interface and Coherency Agent

Interface manages DX100 memory accesses using the **Cache Interface** and **DRAM Interface**. Two implementation options were considered: (a) *Injecting accesses into the LLC:* This ensures memory accesses follow the coherency protocol, benefiting from lower latency and higher LLC bandwidth if they hit. Misses are forwarded to memory controllers by the LLC MSHRs. However, LLC and NoC reorder packets, which can negate the row buffer hit rate improvements achieved by the Indirect Access unit, particularly in mesh-based fabrics like Intel [59, 120]. (b) *Directly accessing memory:* This preserves the order of indirect accesses and avoids LLC MSHR limitations on memory-level parallelism. However, this approach requires the interface to handle coherency. Our design leverages the strengths of both methods. Streaming accesses, which exhibit high spatial and temporal locality, are directed to the LLC using the Cache Interface. Indirect accesses, prone to frequent LLC misses, can bypass the LLC and access memory directly. To maintain coherency, Interface snoops the coherency directories during the fill stage to check if the required cache line is valid, storing this information in the `H` bit of the Row Table. In the request stage, the interface routes the access to the LLC using the Cache Interface if the H bit is set; otherwise, it issues the access directly to memory controllers using the DRAM Interface. This approach ensures correctness, as DX100 maintains exclusive write access to the indirect arrays within the ROI (Section 4.2 – Legality). Thus, no cores can modify the cache lines between snooping and issuing the request.

Interface also contains the **Core Interface** to handle core access to DX100 for sending instructions and reading/writing to the Register File (RF) or Scratchpad (SPD). Figure 6 shows how these memory regions are mapped to the host memory address space, using the system configuration in Table 3. All DX100 memory regions are uncacheable, except for scratchpad data. This region is accessed in a streaming, non-temporal manner. Hence, cache stride prefetchers efficiently prefetch scratchpad data, reducing core access latency. To handle cacheable scratchpad accesses, DX100 includes a **Coherency Agent** that tracks scratchpad cache line status using a single valid bit (`V`) per cache line. When cores read from the scratchpad, the `V` bit is set. When dispatching instructions, DX100 Controller triggers the coherency agent to invalidate all scratchpad cache lines associated with the source and destination tiles of the instruction.

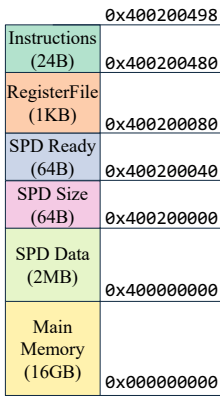


Figure 6: DX100 Memory Regions.

Address Translation. We map the virtual page addresses of DX100 memory regions to the same physical addresses as a general approach for memory-mapped IO communication. We assume the stream and indirect memory regions are mapped through huge pages, which is a common solution for reducing the address translation overhead of applications with large datasets [61, 67, 75, 97].

Using huge pages, we provision a small 256-entry TLB to keep the Page Table Entries (PTE) of the required memory regions. We provide certain DX100 APIs (Section 4.1) to transfer the required PTEs to DX100's TLB once for the whole application lifetime.

4 Programming Model and Compiler

We provide two methods to use DX100 in software: a library of APIs that require manual insertion by the programmer and an automatic compiler system. Due to compiler limitations (memory dependence analysis and code pattern detection), the manual programming method can serve as a fallback and ensure higher code coverage.

4.1 Manual Programming API

Section 3.6 shows that the instructions are transmitted to DX100 using three 64-bit atomic stores targeting specific DX100 memory regions. To facilitate this process, we developed a library of DX100 APIs that perform instruction encoding, memory-mapped accesses, page table entry transfers, and tile/register allocation for various supported data types. Additionally, the library provides a `wait` instruction for core-DX100 synchronization. When invoked, the core will poll the corresponding Scratchpad's ready bit until it is set (refer to Section 3.5). These APIs enable users or compiler infrastructure to seamlessly transform the legacy code for DX100 integration. Figures 7 (a) and (d) show a simple gather code and its offloaded DX100 version using these APIs (noted by **`DX100`**).

Limitations. DX100's support for data access acceleration is limited by the following factors: *Semantic limitation:* Conventional pointer-chasing and linked-list traversal operations involve pointer dereferencing with random base addresses (*e.g.*, `*next[i]`). DX100 does not support this pattern, as our workload analysis did not reveal its usage—likely due to memory fragmentation concerns. For instance, the bucket-chaining join [72] algorithm of Hash-Join benchmark [11] uses array-based indirection (*e.g.*, `nodes[next_idx[i]]`) for linked-list traversal. DX100 accelerates this pattern by processing bulk linked-list traversal operations across many tuples. *Parallelism limitation:* DX100 requires *bulk* accesses for offloading and reordering. In graph workloads [17], the parallelism is determined by the number of nodes in the frontier list. When an iteration involves a few nodes, DX100 acceleration is not beneficial due to the tile under-utilization. In these iterations, we revert to the baseline C code. Across all evaluated graph algorithms, DX100 acceleration covers more than 99% of nodes, while the remaining nodes are processed using the non-accelerated code.

4.2 Compiler

Identifying indirect access patterns and rewriting the whole program using provided APIs could be complicated. Therefore, we propose an MLIR-based compiler to alleviate this problem.

Compiler Transformation Overview. DX100 compiler operates in three stages, as shown by Figure 7. First, it converts the input C/C++ code into MLIR's target agnostic loop-level intermediate representation (`affine`, `scf`) using Polygeist [77], enabling loop-level transformations and analyses. Next, it applies loop tiling to expose opportunities for bulk operations suitable for DX100 offloading.

To identify indirect memory accesses, the compiler employs a Depth First Search (DFS) algorithm [2]. It starts from loop induction variables and traverses the use-def chains to detect indirect access patterns within loops. If indirect access is deemed legal for

```
for(i = 0; i < N; i++)                                   (a)
   v = A[B[i]]
   compute(v)
```

Pass1: Lower from C++ to MLIR, applying tiling

```
for(base = 0; base < N; base += TILE)                    (b)
   for(i = base; i < base + TILE; i++)
      %B_i = load B, i
      %A_B_i = load A, %B_i
      compute(%A_B_i)
```

Pass2: Identify indirections and hoist them out of loop

```
for(base = 0; base < N; base += TILE)                    (c)
   %A_p = alloc_queue(TILE)
   packed_load(i = base; i < base + TILE; i++)
      %B_i = load B, i
      %A_B_i = load A, %B_i
      enqueue %A_p, i, %A_B_i
   for(i = base; i < base + TILE; i++)
      v = dequeue %A_p, i
      compute(v)
```

Pass3: Map packed operations to DX100 APIs

```
// %T0 and %T1 are tile IDs                              (d)
for(base = 0; base < N; base += TILE)
   DX100.SLD(B, %T0, base, base + TILE, 1)
   DX100.ILD(A, %T1, %T0)
   DX100.Wait(%T1)
   for(i = base; i < base + TILE; i++)
      v = load %T1, i
      compute(v)
```

Figure 7: Compiler infrastructure: (a) Legacy gather code, (b) Performing tiling on the single loop, (c) Identifying and hoisting the indirect loads, and (d) Inserting DX100 API calls.

transformation, the compiler hoists indirect memory loads into `packed_load` operations and sinks indirect store and RMW accesses into `packed_store` and `packed_RMW` operations from the innermost loop. As illustrated in Figure 7 (b), the indirect access `A[B[i]]` is hoisted out of the loop into `packed_load`, which is a structured operation that encodes the range of accesses and dependent memory instructions. `A[B[i]]` will be produced by a `packed_load` using `enqueue` operation and consumed by `dequeue` operation in the original single `for` loop. These packed operations serve as high-level abstractions representing bulk memory operations, equivalent to a target-agnostic interface for DAE [115].

During code generation, the compiler pattern-matches for packed operations and generates corresponding DX100 APIs based on their encoded memory dependencies. Since DX100 instructions execute asynchronously with the cores, the compiler inserts a `wait` API call when synchronization is required. This occurs before the core reads data from the scratchpad produced by load, ALU, or range fuser instructions, or when accessing global memory modified by DX100's RMW or store operations. Figure 7 (d) shows that `A[B[i]]` in `packed_load` would be lowered to stream load and indirect load APIs mentioned in Section 4.1.

Legality. Transforming and offloading indirect memory accesses to DX100 requires that no core stores to the memory regions accessed by DX100 within the loop body. Additionally, DX100 acceleration requires no data dependencies between different loop iterations. We leverage MLIR's alias analysis to enforce these requirements. For example, the `Gauss-Seidel` preconditioner used in multigrid

solvers [43, 66] performs indirect loads from a data array while also storing to the array at different indices. While this workload involves substantial indirect memory accesses, DX100 cannot accelerate it because any aliasing between load and store indices could result in stale data if indirect loads are hoisted outside the loop.

5 Evaluation Methodology

Benchmarks.DX100 is first evaluated under different cache conditions and DRAM access patterns using five microbenchmarks. Then, 12 workloads with indirect access patterns from five benchmark suites are evaluated for the main evaluation: (1) Conjugate Gradient (CG) of a $150K \times 150K$ matrix and Integer Sort (IS)[1] of 2^{25} keys from the NAS parallel benchmark suite [7], representative of the computational fluid dynamics applications. (2) GZ, GP, GZI, and GZI workloads from the UME proxy [42], which perform a gradient computation over 2M zones and points of an unstructured mesh used in hydrodynamic simulations. (3) Spatter benchmark [64] using an indirect access pattern collected using the methodology described in [109] from the xRAGE parallel multi-physics application [35]. (4) Breadth-First Search (BFS)[1], PageRank (PR), and Betweenness-Centrality (BC) graph algorithms from GAP benchmark suite [17] using a uniform graph with 2^{20} to 2^{22} nodes and an average degree of 15. (5) Two implementations of the Parallel Radix Join algorithms from the Hash-Join benchmark suite [11], representative of the in-memory database workloads: histogram-based (PRH) [56] and bucket-chaining-based (PRO) [72] using 2M tuples.

Simulation Infrastructure. DX100 is integrated as a memory-mapped accelerator to the execution-based and event-driven Gem5 simulator [21] equipped with Ramulator2 [71] as the backend memory simulator. This setup accurately models both the core and memory access. DX100 is evaluated using a 4-core shared-memory system with configurations similar to the Intel Skylake architecture [59, 120] as shown by Table 3. *For a fair comparison with the baseline due to the scratchpad and area overhead of DX100, we increase the LLC size of the baseline by 2MB.* A functional simulator for DX100 APIs (see Section 4.1) was developed to ensure the correctness of the implementations before simulation. Correctness is re-verified with the Gem5 simulation.

Table 3: System Configuration

Component	Configuration
Core	4 8-wide cores, LQ: 72, SQ: 56, IQ: 50, ROB: 224, 180 Int Registers, 168 Vector/Floating-Point Registers, 3.2GHz Frequency
L1I cache	32KB, 8-way, 2 cycles latency, 16 MSHRs, Stride Prefetcher
L1D cache	32KB, 8-way, 4 cycles latency, 16 MSHRs, Stride Prefetcher
L2 cache	256KB, 4-way, 12 cycle latency, 32 MSHRs, Stride Prefetcher
LLC	**Baseline/DMP:** 10MB, 20-way, **DX100**: 8MB, 16-way 42 cycle latency, 256 MSHRs
Memory	2 Channels, DDR4 3200, 51.2GB/s Max BW, t_{CK}=625ps $t_{RP/RCD}$=12.5ns, $t_{CCDS/L}$=2.5/5.0ns, t_{RTP}=7.5ns, t_{RAS}=32.5ns Request Buffer size: 32/channel, FR-FCFS scheduler
DX100	2MB Scratchpad w/ 4 ports: $32 \times 16K$ TILEs, 64×8 Row Table slices 32 Registers, 128 Request Table size, 16 ALU lanes, 256-entry TLB

Indirect Prefetcher Modeling. We compare the performance and bandwidth utilization of DX100 with the state-of-the-art indirect prefetcher, DMP [33], using their public Gem5 artifact [23]. We reproduced DMP's performance results using their best-performant workload, IS, and their *single-core baseline* configuration with *256KB last-level (L2) cache* size. Then, we used the *4-core* baseline configuration (Table 3) with *10MB L3 cache* for comparison with DX100. Compared to the reported results [33], we observed less performance improvement for DMP with larger caches and more cores in our baseline configuration.

Power and Area Modeling. DX100 was implemented in RTL and all components were synthesized with Synopsys Design Compiler using 28nm TSMC library. BCAM area and power for the Row Table is evaluated in 28nm FDSOI technology [52]. The baseline Skylake core area is evaluated using die shots [125]. To compare DX100 area with the baseline core, all area and power numbers are scaled to 14nm using equations of [118].

[1]We follow prior work by evaluating bottom-up BFS and disabling buckets for IS. DX100 can also accelerate data access in top-down BFS and bucket-based IS algorithms.

6 Results

6.1 Microbenchmarks

To evaluate DX100 benefits under certain index distributions, we implement five microbenchmarks with Gather, Scatter, and RMW access patterns under two scenarios.

Scenario 1: All-Hits. In this scenario, we warm up all caches and use a streaming index distribution (`B[i] = i`), enabling cores to fetch all data directly from the L1 cache, thus there are no benefits from memory access reordering for this scenario. This setup highlights the ability to accelerate address calculation operations and reduce the dynamic instruction count. Figure 8 (a) illustrates the performance gains of DX100 over the baseline.

In *Gather-SPD*, we only offload the gather operation (`p_A[i] = A[B[i]]`) to DX100. The packed array (`p_A[i]`) is loaded from the scratchpad (SPD) to the cores. While DX100 reduces the dynamic instruction by 2.9×, loading the packed array from the higher latency SPD limits the overall performance speedup to 1.2×. The SPD latency impacts performance minimally when we move to realistic benchmarks with higher cache misses and memory accesses.

In the *Gather-Full* experiment, the entire kernel (`C[i]=A[B[i]]`) is offloaded to DX100 by using a streaming store operation on `C[i]`, achieving a 3.2× performance gain by reducing CPU core's dynamic instructions from 870K in the baseline to just 273 with DX100.

For RMW operations, the baseline implementations rely on atomic accesses to ensure correctness, which is ≈4.8× slower than non-atomic operations because of the memory fences [4] and cache-line or bus locking mechanisms [48]. As a shared accelerator and the sole writer to indirect memory regions (Section 4.2), DX100 eliminates fine-grain atomic operations. To assess DX100 benefits for RMW (`A[B[i]] += C[i]`), we compare it to two baseline implementations: one using atomic operations (*RMW-Atomic*) and another ignoring correctness (*RMW-NoAtom*). DX100 outperforms these baselines by 17.8× and 3.7×, respectively, demonstrating high efficiency for common RMW operations (Table 1).

Scatter operations (`A[B[i]] = C[i]`) cannot be parallelized in a shared memory system even with atomic operations as reordering the indirect stores results in WAW data hazards and compromises correctness. Therefore, we use a single-core configuration with a 4MB LLC for the baseline and a 2MB LLC for DX100. In this configuration, DX100 improves the scatter performance by 6.6×.

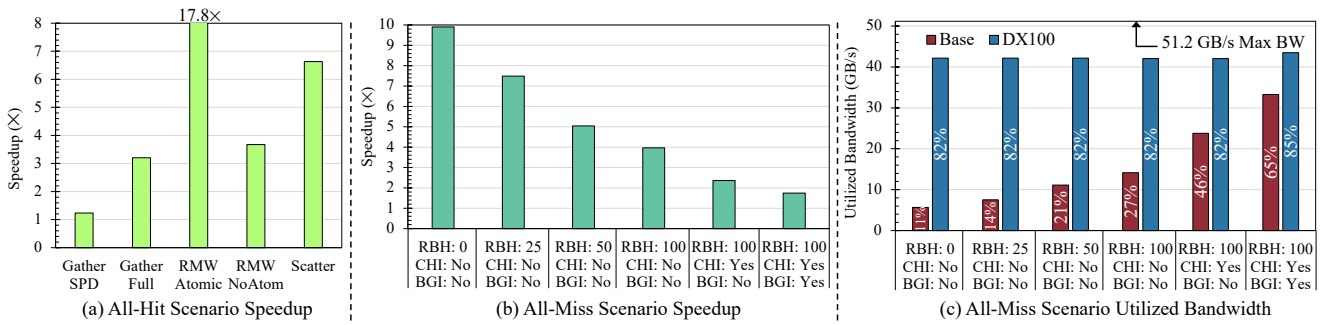


Figure 8: (a) DX100 speedup for different access types under 100% L1 cache hit rate, (b) DX100 speedup and (c) utilized bandwidth for Gather-Full kernel w.r.t. row buffer hit rate (RBH), channel interleaving (CHI), and bank-group interleaving (BGI).

This speedup is 2× higher than Gather-Full primarily because the baseline is restricted to one core due to data hazards.

Scenario 2: All-Misses. To evaluate the memory bandwidth improvements of DX100, we analyze an All-Miss scenario using the *Gather-Full* microbenchmark. In this setup, all indirect accesses miss the cache, requiring data to be fetched directly from DRAM. We generate a constant set of 64K unique `B[i]` indices to evenly distribute the indirect `A[B[i]]` words across 16 rows in all banks, bank groups, and channels. These unique indices are then reordered to artificially create various **row-buffer hit rates (RBH)** (0%-100%), **channel interleaving (CHI)**, and **bank-group interleaving (BGI)** patterns between consecutive indices.

Figures 8 (b) and (c) present the performance and bandwidth utilization of DX100 compared to the baseline. The row-buffer hit rates (RBH) and memory-level parallelism (CHI, BGI) increase in the benchmark access patterns from left (worst) to right (best); DX100 performance improvements decrease from left to right, with a maximum improvement of 9.9× with the worst-case index pattern.

Several key observations are evident. *First*, DX100 consistently achieves high bandwidth utilization, ranging from 82% to 85%, regardless of the order of the indices in the input dataset. This is due to DX100's ability to reorder memory addresses. In contrast, the baseline relies on DRAM memory controllers, which have limited reordering capabilities due to limited visibility into future memory requests. *Second*, even with the best-case index ordering for the baseline with 100% RBH, CHI, and BGI (right-most bar), it achieves only 65% of the maximum 51.2GB/s bandwidth because of its limited memory-level parallelism. In this configuration, DX100 achieves a 1.7× speedup, primarily because of its higher memory access rate. *Third*, with no BGI (second right-most bar), the theoretical bandwidth utilization drops to half as the column-to-column read is constrained by t_{CCDL}. Here, the baseline utilizes 46% of DRAM bandwidth. When CHI is further eliminated, the baseline channel utilization drops from 46% to 27%, effectively leaving one channel idle. Finally, as the row-buffer hit rate decreases from 100% to 0% (left-most bars), the baseline's bandwidth utilization drops by 2.5×.

6.2 Performance Analysis

Figure 9 shows that DX100 achieves a geometric mean speedup of 2.6× compared to the baseline across 12 irregular workloads. This speedup is due to significantly higher bandwidth utilization, instruction reduction, and cache miss reductions. In the following paragraphs, we discuss each metric in detail.

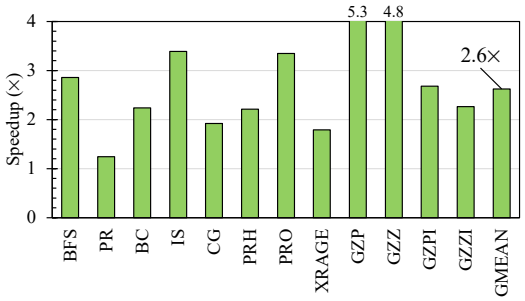
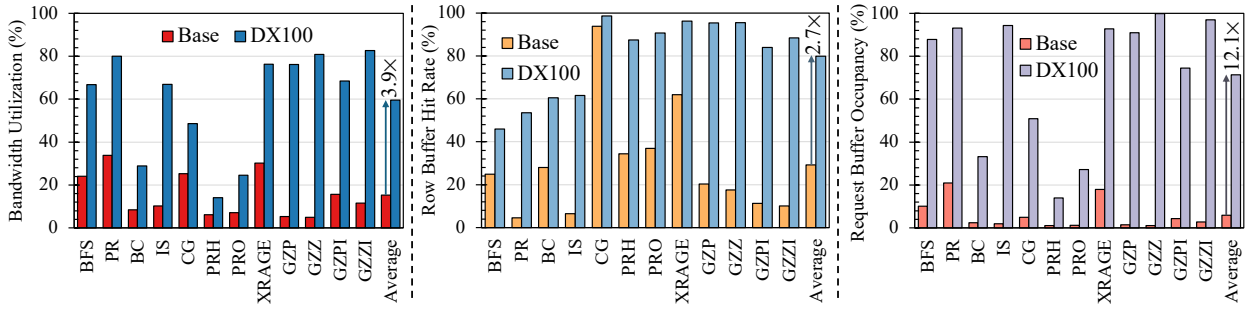


Figure 9: DX100 speedup for different workloads.

Higher Bandwidth Utilization. Figure 10 (a) compares the bandwidth utilization of DX100 with the baseline, showing an average improvement of 3.9×. The bandwidth improvement is particularly higher in workloads with significant indirect memory accesses, as DX100 enhances the efficiency of these accesses through reordering, coalescing, and interleaving techniques. For instance, the *Integer Sort* (`IS`) kernel from the `NAS` benchmark suite involves indirect indexing to a large key array containing 2^{25} elements. Here, indirect accesses consume a significant portion of the memory bandwidth, and DX100 improves the bandwidth utilization by 6.5×. Conversely, the Conjugate Gradient (`CG`) kernel from the same benchmark suite operates on a sparse matrix format, where most memory accesses involve streaming to the matrix, with relatively fewer indirect accesses targeting a vector. Consequently, DX100 achieves a lower improvement of 1.9× in bandwidth utilization for this kernel.

DX100 improvement of memory bandwidth utilization for indirect accesses is primarily attributed to two key factors:

First, *DX100 significantly improves the DRAM row-buffer hit rate by reordering memory accesses*, achieving an average increase of 2.7×, as shown in Figure 10 (b). The row-buffer hit rate is defined as the proportion of memory accesses that find their target column in the row-buffer. DX100 substantially enhances the row-buffer hit rate for UME kernels, *i.e.,* `GZP`, `GZZ`, `GZPI`, and `GZZI`. Analyzing the UME dataset, consisting of 2M data points, reveals an average index distance ($abs(i - B[i])$) of 85K elements, indicating limited spatial locality in the data. Despite this, DX100's reordering technique within a tile of 16K elements successfully grouped 7.6 column accesses per DRAM row, allowing the indirect access unit to issue these accesses



(a) Bandwidth Utilization (b) DRAM Row Buffer Hit Rate (c) DRAM Controller Request Buffer Occupancy

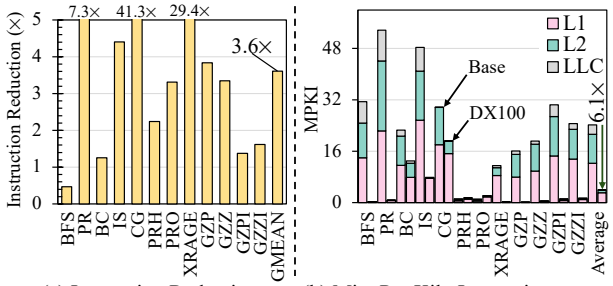
Figure 10: (a) Bandwidth utilization, (b) row buffer hit rate, and (c) request buffer occupancy of baseline vs. DX100.

consecutively. This optimization improves the row-buffer hit rate of UME kernels by 6.1×, from 15% to 91%.

Second, *DX100 increases the memory-level parallelism and access rate*. To evaluate this claim, we measure the request buffer occupancy, defined as the ratio of the average number of memory accesses in the DRAM memory controller's request buffer to the maximum buffer size (32). Occupancy is a critical factor for DRAM controllers, enabling the interleaving of commands across different banks and hiding the latency of `PRE` and `ACT` commands for one bank behind the latency of reads and writes to another bank. DX100 improves the request buffer occupancy of the baseline by 12.1× because it breaks the dependency between index loads and indirect accesses, enabling bulk memory access. Furthermore, DX100 is placed near the memory controllers, bypassing the structural limitations in the core and memory system. On the other hand, the baseline suffers from limited outstanding memory accesses because only 54 instructions in a 224-entry ROB are loads and stores across all cycles of all benchmarks. These load and store instructions generate a chain of dependencies with prior memory accesses and address calculation instructions [14]. On average, 19 outstanding loads in the LQ are issued to the memory system at any given cycle. From these, only 10 accesses target data arrays, and the rest are stack accesses or register spills and fills, which often hit the L1 cache. Finally, many word accesses are coalesced in the MSHRs and hit in the cache. Together, these factors reduce the request buffer occupancy of the baseline to an average of only 2 memory accesses, highlighting the substantial improvements enabled by DX100.

Instruction and Cache Miss Reduction. Figure 11 (a) shows that DX100 reduces the number of core instructions by a geometric mean of 3.6×, which can significantly improve CPU core energy consumption. The reduction in dynamic instruction count is achieved primarily by accelerating indirect address calculation operations. However, `BFS` instruction count slightly increases in DX100 implementation. This is due to the use of OpenMP `critical` primitives for synchronization between cores accessing DX100, which rely on spinning locks. These locks increase the instruction count while waiting for another core to complete its DX100 instructions.

Despite fewer instructions, Figure 11 (b) shows that DX100 significantly reduces the cache MPKI, measured across the whole workload execution. This improvement is because indirect accesses in the baseline often poorly utilize cache lines, leading to cache pollution and wasted capacity. In contrast, DX100 directly injects these accesses into memory, packs them, and writes the packed



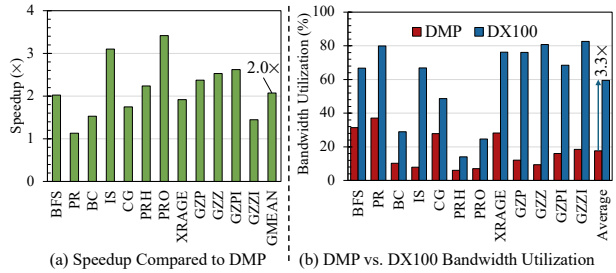
(a) Instruction Reduction (b) Miss Per Kilo Instructions (MPKI)

Figure 11: DX100 (a) instruction and (b) MPKI reduction.

data to the Scratchpad. Cores access this packed data in a streaming manner from the Scratchpad, preserving cache space. Additionally, intermediate index arrays are accessed exclusively by DX100, freeing up private cache space for cores. As a result, DX100 effectively increases the usable cache capacity for critical core accesses.

6.3 Comparison with Indirect Prefetcher

Figure 12 (a) shows that DX100 outperforms the state-of-the-art indirect prefetcher, DMP [33], achieving a 2.0× geometric mean speedup. This is due to DX100's 3.3× higher bandwidth utilization, as depicted in Figure 12 (b). While DMP improves baseline bandwidth by increasing memory access rates, it *does not reorder* memory accesses and primarily relies on the memory controllers for DRAM command optimization. In contrast, DX100's bulk accesses surpass the limited future visibility of memory controllers, enabling better bandwidth utilization with effective reordering.



(a) Speedup Compared to DMP (b) DMP vs. DX100 Bandwidth Utilization

Figure 12: (a) Speedup and (b) Bandwidth Utilization of DX100 compared to DMP [33].

DX100 reduces dynamic instructions and improves energy consumption of CPU cores, unlike indirect prefetchers that leave the instruction footprint unchanged. Indirect prefetchers primarily lower memory access latency by improving cache hit rates; for instance, DMP reduces the average memory access latency by an average of 1.4× over the baseline. However, accurate prefetching is challenging for irregular workloads with conditional accesses. Many of the evaluated workloads involve conditional accesses (Table 1) that are difficult for prefetchers to handle effectively. Prefetching untaken loop iterations degrades performance by polluting the cache, especially with low cache line utilization of indirect accesses. In contrast, DX100 ensures 100% accurate fetching through a programmable interface supporting ALU operations and conditional accesses.

6.4 Sensitivity Analysis

Figure 13 shows that increasing tile size from 1K to 32K elements improves the performance from 1.7× to 2.9× over the baseline. This speedup arises from two primary factors. *First,* DX100 reduces memory accesses by coalescing redundant addresses within larger tiles; 32K tile size reduces memory accesses by a geometric mean of 1.4× compared to a 1K tile size. *Second,* DX100 increases memory bandwidth utilization by 25% when enlarging the tile size from 1K to 32K, primarily due to a 27% higher row-buffer hit rate with a 32K tile size. Notably, increasing the tile size has minimal impact on the memory access rate (DRAM controller occupancy), as this is primarily determined by the number of outstanding memory accesses, which remains unchanged by the tile size.

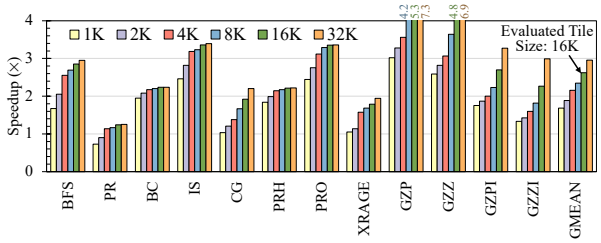


Figure 13: Performance sensitivity to the tile size.

6.5 Area and Power Analysis

Table 4 provides a detailed area and power breakdown for all DX100 components, synthesized using a 28 *nm* library. The area and power are dominated by the Scratchpad, which contains 32 tiles of 16K elements, facilitating the communication between DX100 components and cores. Using die shots of a 4-core Skylake processor [125] fabricated in a 14nm technology node, we estimate the area of a core to be approximately $10.1mm^2$. Of this, around $2.3mm^2$ is occupied by a 2MB cache slice, which includes data arrays, tag arrays, and coherency directory. Applying the scaling factors from [118], we estimate DX100 area in 14nm to be approximately $1.5mm^2$ which results in a 3.7% overhead for the processor as DX100 is shared between four cores. Since DX100 area is comparable to the area of a single cache slice in the baseline architecture, *all evaluations in this paper use a baseline with a 2MB larger LLC* (see Table 3). Our evaluation shows around $777mW$ power consumption for DX100.

Table 4: DX100 Area and Power Analysis in 28 *nm*

Module	Area (mm^2)	Power (mW)
Range Fuser	0.001	0.26
ALU	0.095	74.83
Stream Access	0.012	6.03
Indirect Access	0.323	83.70
Controller	0.002	0.43
Interface	0.045	30.0
Coherency Agent	0.010	3.12
Register File	0.005	1.56
Scratchpad	3.566	577.03
Total	4.061	777.17

6.6 Scalability Discussion

Our evaluation in prior sections focused on a system with four cores and two DDR4-3200 memory channels. When scaling the System-on-Chip (SoC) with more cores and memory channels, multiple instances of DX100 can be integrated to utilize the increased memory bandwidth. We explored two approaches for integrating multiple DX100 instances as discussed next.

(1) Address range partitioning: Assigns a DX100 instance to a specific memory address range or region. The programmer or compiler ensures that an entire data structure or array is allocated within a single region, guaranteeing exclusive access by a single DX100 instance. This exclusivity ensures the correctness of RMW operations without additional design complexity. However, if interacting arrays are assigned to different regions, scratchpad tile transfers between DX100 instances become necessary. While allocating all interacting data structures to the same region can eliminate these transfers, it may also lead to load imbalance among DX100 instances limiting scalability.

(2) Core multiplexing: Assigns multiple cores to a DX100 instance for acceleration, ensuring that all DX100 instructions from a given core are directed to a specific instance. This design places DX100 instances closer to their associated cores within the SoC, reducing latency for core-scratchpad accesses. To maintain correctness for RMW operations, only one instance must have exclusive write access to an indirect array. This is enforced through a *coarse-grained region-based coherence protocol* [78], which upholds the Single-Writer-Multiple-Reader (SWMR) invariant [116] among instances. DX100 receives address ranges from page table entry transfers and maintains them in the TLB (Section 3.6). When DX100 dispatches memory access instructions, the entire address range covering the array (`&A[0]` to `&A[SIZE]`) is treated as a coherence region. The Coherency Agent verifies the coherence state and issues a coherence request if the region is not in the required state. Memory access instructions are issued once the appropriate state is acquired. During execution, the coherence region is locked to prevent other instances from obtaining write permission. This protocol operates independently of the primary multicore coherence mechanism, as DX100 assumes that cores do not simultaneously modify arrays (Section 4.2 – Legality).

Due to the overhead of inter DX100 tile transfers and load imbalance limitations of the address range partitioning, we implement the second approach, core multiplexing, in Gem5. We study the performance scalability by doubling the cores, LLC size, memory channels, and the input dataset size of the benchmarks. We evaluate two DX100 configurations: one with a single instance but a 4MB

scratchpad shared by eight cores, and another with two instances each with a 2MB scratchpad and shared by four cores.

Figure 14 presents the performance scalability results normalized to baselines with the same core count, highlighting two key takeaways. First, DX100 sustains its performance advantage over the baseline system as the core count scales to eight, achieving a geometric mean speedup of 2.6× with four cores (green bars) and 2.5× with eight cores (orange bars). A single DX100 instance utilizes 60% of the available bandwidth across two memory channels (51.2 GB/s peak) and 54% across four memory channels (102.4 GB/s peak). Second, the core multiplexing approach does not introduce significant bottlenecks from additional coarse-grained region-based coherence messages and checks for multiple instances of DX100. Despite this overhead, integrating two DX100 instances even increases speedup to 2.7× (blue bars) due to the use of additional functional units.

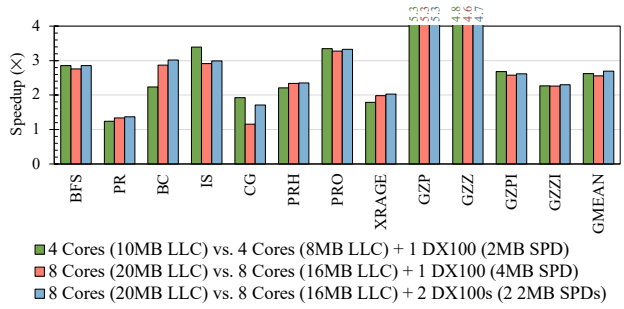


Figure 14: Performance improvement scalability using different number of cores and DX100 instances.

7 Related Work

Efficient data access in irregular applications is a longstanding challenge due to the prevalence of indirect memory accesses. Here, we discuss many approaches to mitigate the memory access bottleneck.

Prefetcher. Software prefetching [2, 51, 55, 68, 79, 80] aims to hide memory latency by inserting prefetch instructions for data likely to be accessed soon. These techniques leverage static analysis, runtime profiling, or programmer annotations to detect indirect loads. Hardware prefetchers [6, 8, 9, 18, 19, 27, 33, 46, 50, 53, 62, 76, 91, 110, 124, 126, 131] predict future accesses at runtime by exploiting spatial and temporal locality. Hybrid approaches [3, 100, 119, 129] combine compiler or programmer annotations with hardware support to improve prefetching accuracy. Runahead execution [29, 84, 86–90, 104] allows the processor to speculatively execute instructions when it is stalled due to long-latency memory accesses. Prefetching effectively reduces average memory access latency but provides limited improvements to memory bandwidth utilization. In contrast, DX100 enhances the memory bandwidth through reordering, coalescing, and interleaving. Prefetchers do not impact the instruction footprint, while DX100 accelerates address calculation operations. Moreover, prefetchers suffer from low accuracy with conditional memory accesses, leading to cache pollution and wasting the cache capacity. DX100 supports conditional operations and injects the indirect accesses to the DRAM, improving cache line utilization and reducing the cache misses for other critical data accesses of the core.

Fetcher Units, inspired by Decoupled Access-Execute paradigm [115], aim to improve performance by separating and offloading programs into independent access and execute cores. This decoupling allows memory accesses and computations to be processed concurrently on dedicated processors, hiding memory latency. Fetcher units serve as the access core to perform specific memory accesses and feed the data to the processor to avoid recomputation. Specialized fetchers have been proposed for various domains. HATS [81] performs locality-aware graph traversals to reduce data movement. SPU [28], TMU [114] and MAPLE [93] can support traversal of sparse tensors and processing of sparse linear algebra. Widx [58] accelerates hash indexing operations. SQRL [60] handles traversals of hash tables and tree data structures. General-purpose fetcher units such as SpZip [130] and Terminus [65] support a wider range of applications and exploit fine-grained pipelining to hide the latency of indirect memory access chains. SpZip incorporates data compression to reduce memory traffic, while Terminus supports fine-grained updates for recursive data structures like trees and linked lists. However, fetchers provide insufficient visibility into future memory accesses, which hinders effective memory access reordering and coalescing. DX100 differs by providing a programmable accelerator capable of improving memory bandwidth utilization for bulk memory accesses by reordering indices.

Specialized Accelerators have been proposed to offload both data access and computation for irregular workloads. Near-memory processing architectures [1, 39, 44, 54, 92, 127] and Near-cache processing architectures [10, 82, 106, 117, 132, 133] place computation close to memory or cache to reduce the memory access latency that is imposed by data movement. While DX100 also reduces the data transfer overhead of indirection, its primary benefit is from enhancing the memory bandwidth utilization. Additionally, some accelerators [5, 37, 41, 83, 94, 105, 122, 123] employ customized architectures and reconfigurable data-flow designs to handle irregular data access patterns and exploit parallelism effectively. These specialized accelerators typically support a narrow set of applications and require substantially redesigning existing algorithms or data structures to fit their specific hardware models. In contrast, DX100 offers a general-purpose solution that supports multiple degrees of indirection, diverse loop patterns, and various access types without significantly changing existing algorithms.

Memory access reordering. DRAM memory controllers [47, 49, 73, 85, 102, 103, 108] have been designed to reorder requests to minimize page misses, bank conflicts, and improve row-buffer hit rates. However, they can schedule requests only within a limited window of visibility—typically around 48 cache lines per memory channel [98]—which restricts reordering opportunities. To exploit reordering beyond this window, software solutions like Milk [57] and Propagation Blocking [16] first collect random indirect memory accesses into cache-fitting batches and then reorder them into efficient sequential DRAM accesses. Propagation Blocking specifically accelerates PageRank, while Milk is a compiler that supports a broad set of applications with programmer annotations. However, these software solutions introduce timing and space overhead due to the need to introduce intermediate data structures, which can pollute the cache and sacrifice temporal locality. Hardware approaches like PHI [82] and COBRA [10] modify the cache hierarchy to defer writes or read-modify-write operations with poor

spatial locality, batching them to achieve sequential main memory accesses. Crescent [31] co-designs an approximate neighbor search algorithm and a hardware accelerator to convert irregular DRAM accesses to sequential DRAM accesses for deep point cloud analytics workloads. GSDRAM [107] supports access with non-unit strides on commodity DRAM to improve bandwidth utilization but does not optimize indirect accesses. In contrast, DX100 offers several advantages over these approaches. Compared to memory controllers, DX100 provides a much larger window for reordering memory requests, enabling more effective memory bandwidth utilization. Unlike software solutions, DX100 operates at the hardware level without introducing space overhead or causing cache pollution from intermediate data structures. Moreover, compared to existing hardware solutions, DX100 can support complex and general-purpose indirect memory access patterns.

8 Conclusion

DX100 is a programmable data access accelerator designed to optimize memory bandwidth utilization for irregular applications using two ideas. First, DX100 is placed near the memory controllers, allowing it to inject memory accesses at a higher rate and bypass the core and memory system limitations. Second, DX100 employs three key techniques over bulk indices—*reordering*, *coalescing*, and *interleaving*—to improve DRAM row-buffer hit rate, reduce memory accesses, and optimize the bandwidth of DRAM channels and bank-groups. Using DX100's general-purpose ISA with 8 instructions, we accelerate 12 benchmarks accross five benchmark suites, and observe 2.6× performance improvement, 3.9× higher bandwidth utilization, and 3.6× instruction reduction.

References

[1] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. A scalable processing-in-memory accelerator for parallel graph processing. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*. 105–117.

[2] Sam Ainsworth and Timothy M. Jones. 2017. Software prefetching for indirect memory accesses. In *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 305–317. doi:10.1109/CGO.2017.7863749

[3] Sam Ainsworth and Timothy M Jones. 2018. An event-triggered programmable prefetcher for irregular workloads. *ACM Sigplan Notices* 53, 2 (2018), 578–592.

[4] Ashkan Asgharzadeh, Juan M. Cebrian, Arthur Perais, Stefanos Kaxiras, and Alberto Ros. 2022. Free atomics: hardware atomic operations without fences. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 14–26. doi:10.1145/3470496.3527385

[5] Daehyeon Baek, Soojin Hwang, Taekyung Heo, Daehoon Kim, and Jaehyuk Huh. 2021. Innersp: A memory efficient sparse matrix multiplication accelerator with locality-aware inner product processing. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 116–128.

[6] Jean-Loup Baer and Tien-Fu Chen. 1991. An effective on-chip preloading scheme to reduce data access penalty. In *Proceedings of the 1991 ACM/IEEE conference on Supercomputing*. 176–186.

[7] David H Bailey, Eric Barszcz, John T Barton, David S Browning, Robert L Carter, Leonardo Dagum, Rod A Fatoohi, Paul O Frederickson, Thomas A Lasinski, Rob S Schreiber, et al. 1991. The NAS parallel benchmarks—summary and preliminary results. In *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing*. 158–165.

[8] Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2018. Domino temporal data prefetcher. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 131–142.

[9] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo spatial data prefetcher. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 399–411.

[10] Vignesh Balaji and Brandon Lucia. 2022. Improving Locality of Irregular Updates with Hardware Assisted Propagation Blocking.. In *HPCA*. 543–557.

[11] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 362–373. doi:10.1109/ICDE.2013.6544839

[12] Ronald Barber, Guy Lohman, Ippokratis Pandis, Vijayshankar Raman, Richard Sidle, Gopi Attaluri, Naresh Chainani, Sam Lightstone, and David Sharpe. 2014. Memory-efficient hash joins. *Proceedings of the VLDB Endowment* 8, 4 (2014), 353–364.

[13] Abanti Basak, Shuangchen Li, Xing Hu, Sang Min Oh, Xinfeng Xie, Li Zhao, Xiaowei Jiang, and Yuan Xie. 2019. Analysis and Optimization of the Memory Hierarchy for Graph Processing Workloads. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 373–386. doi:10.1109/HPCA.2019.00051

[14] Abanti Basak, Shuangchen Li, Xing Hu, Sang Min Oh, Xinfeng Xie, Li Zhao, Xiaowei Jiang, and Yuan Xie. 2019. Analysis and Optimization of the Memory Hierarchy for Graph Processing Workloads. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 373–386. doi:10.1109/HPCA.2019.00051

[15] Scott Beamer, Krste Asanovic, and David Patterson. 2012. Direction-optimizing Breadth-First Search. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–10. doi:10.1109/SC.2012.50

[16] Scott Beamer, Krste Asanović, and David Patterson. 2017. Reducing pagerank communication via propagation blocking. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 820–831.

[17] Scott Beamer, Krste Asanović, and David Patterson. 2017. The GAP Benchmark Suite. arXiv:1508.03619 [cs.DC] https://arxiv.org/abs/1508.03619

[18] Michael Bekerman, Stephan Jourdan, Ronny Ronen, Gilad Kirshenboim, Lihu Rappoport, Adi Yoaz, and Uri Weiser. 1999. Correlated load-address predictors. *ACM SIGARCH Computer Architecture News* 27, 2 (1999), 54–63.

[19] Rahul Bera, Anant V Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. Dspatch: Dual spatial pattern prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 531–544.

[20] Marsha J Berger and Joseph Oliger. 1984. Adaptive mesh refinement for hyperbolic partial differential equations. *J. Comput. Phys.* 53, 3 (1984), 484–512. doi:10.1016/0021-9991(84)90073-1

[21] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 simulator. *SIGARCH Comput. Archit. News* 39, 2 (Aug. 2011), 1–7. doi:10.1145/2024716.2024718

[22] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality*. *The Journal of Mathematical Sociology* 25, 2 (2001), 163–177. doi:10.1080/0022250X.2001.9990249

[23] Institute of AI & Robotics of Xi'an Jiaotong Univiersity CAG group. 2024. *A gem5 experimental repo in order to explore Data-dependent Access (DDA)*. https://github.com/xjtuiair-cag/gem5_dda/tree/dmp-paper

[24] Yuhan Chen, Alireza Khadem, Xin He, Nishil Talati, Tanvir Ahmed Khan, and Trevor Mudge. 2023. PEDAL: A Power Efficient GCN Accelerator with Multiple DAtafLows. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. doi:10.23919/DATE56975.2023.10137240

[25] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, et al. 2014. Dadiannao: A machine-learning supercomputer. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 609–622.

[26] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 127–138. doi:10.1109/JSSC.2016.2616357

[27] Yuan Chou. 2007. Low-cost epoch-based correlation prefetching for commercial applications. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. IEEE, 301–313.

[28] Vidushi Dadu, Jian Weng, Sihao Liu, and Tony Nowatzki. 2019. Towards general purpose acceleration by exploiting common data-dependence forms. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 924–939.

[29] James Dundas and Trevor Mudge. 1997. Improving data cache performance by pre-executing instructions under a cache miss. In *Proceedings of the 11th International Conference on Supercomputing* (Vienna, Austria) *(ICS '97)*. Association for Computing Machinery, New York, NY, USA, 68–75. doi:10.1145/263580.263597

[30] R D Falgout, J E Jones, and U M Yang. 2004. The Design and Implementation of hypre, a Library of Parallel High Performance Preconditioners. *Lecture Notes in Computational Science and Engineering* 51 (7 2004). https://www.osti.gov/biblio/875356

[31] Yu Feng, Gunnar Hammonds, Yiming Gan, and Yuhao Zhu. 2022. Crescent: taming memory irregularities for accelerating deep point cloud analytics. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 962–977.

[32] Bruce Fleischer, Sunil Shukla, Matthew Ziegler, Joel Silberman, Jinwook Oh, Vijavalakshmi Srinivasan, Jungwook Choi, Silvia Mueller, Ankur Agrawal, Tina Babinsky, et al. 2018. A scalable multi-TeraOPS deep learning processor core for AI trainina and inference. In *2018 IEEE symposium on VLSI circuits*. IEEE, 35–36.

[33] Gelin Fu, Tian Xia, Zhongpei Luo, Ruiyang Chen, Wenzhe Zhao, and Pengju Ren. 2024. Differential-Matching Prefetcher for Indirect Memory Access. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 439–453. doi:10.1109/HPCA57654.2024.00040

[34] Michael Gittings, Robert Weaver, Michael Clover, Thomas Betlach, Nelson Byrne, Robert Coker, Edward Dendy, Robert Hueckstaedt, Kim New, W Rob Oakes, Dale Ranta, and Ryan Stefan. 2008. The RAGE radiation-hydrodynamic code. *Computational Science & Discovery* 1, 1 (Nov. 2008), 015005. doi:10.1088/1749-4699/1/1/015005

[35] John W. Grove. 2019. Eulerian Applications Project - xRage Introduction and Overview. (5 2019). doi:10.2172/1532688

[36] Yufeng Gu, Arun Subramaniyan, Tim Dunn, Alireza Khadem, Kuan-Yu Chen, Somnath Paul, Md Vasimuddin, Sanchit Misra, David Blaauw, Satish Narayanasamy, and Reetuparna Das. 2023. GenDP: A Framework of Dynamic Programming Acceleration for Genome Sequencing Analysis. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 25, 15 pages. doi:10.1145/3579371.3589060

[37] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. 2016. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *2016 49th annual IEEE/ACM international symposium on microarchitecture (MICRO)*. IEEE, 1–13.

[38] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A Horowitz, and William J Dally. 2016. EIE: Efficient inference engine on compressed deep neural network. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 243–254.

[39] Milad Hashemi, Khubaib, Eiman Ebrahimi, Onur Mutlu, and Yale N. Patt. 2016. Accelerating Dependent Cache Misses with an Enhanced Memory Controller. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 444–455. doi:10.1109/ISCA.2016.46

[40] Xin He, Subhankar Pal, Aporva Amarnath, Siying Feng, Dong-Hyeon Park, Austin Rovinski, Haojie Ye, Yuhan Chen, Ronald Dreslinski, and Trevor Mudge. 2020. Sparse-TPU: Adapting systolic arrays for sparse matrices. In *Proceedings of the 34th ACM international conference on supercomputing*. 1–12.

[41] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W Fletcher. 2019. Extensor: An accelerator for sparse tensor algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 319–333.

[42] Paul Henning and USDOE National Nuclear Security Administration. 2023. Ume: Unstructured Mesh Explorations. doi:10.11578/dc.20230602.5

[43] Michael Allen Heroux, Jack Dongarra, and Piotr Luszczek. 2013. *HPCG Benchmark Technical Specification*. Technical Report. Sandia National Lab. (SNL-NM), Albuquerque, NM (United States). doi:10.2172/1113870

[44] Byungchul Hong, Gwangsun Kim, Jung Ho Ahn, Yongkee Kwon, Hongsik Kim, and John Kim. 2016. Accelerating linked-list traversal through near-data processing. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation*. 113–124.

[45] Mark Horowitz. 2014. 1.1 Computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 10–14. doi:10.1109/ISSCC.2014.6757323

[46] Zhigang Hu, Margaret Martonosi, and Stefanos Kaxiras. 2003. TCP: Tag correlating prefetchers. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings*. IEEE, 317–326.

[47] Ibrahim Hur and Calvin Lin. 2004. Adaptive history-based memory schedulers. In *37th International Symposium on Microarchitecture (MICRO-37'04)*. IEEE, 343–354.

[48] Intel. 2025. *Intel® 64 and IA-32 Architectures Software Developer Manuals*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[49] Engin Ipek, Onur Mutlu, José F Martínez, and Rich Caruana. 2008. Self-optimizing memory controllers: A reinforcement learning approach. *ACM SIGARCH Computer Architecture News* 36, 3 (2008), 39–50.

[50] Akanksha Jain and Calvin Lin. 2013. Linearizing irregular memory accesses for improved correlated prefetching. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*. 247–259.

[51] Saba Jamilan, Tanvir Ahmed Khan, Grant Ayers, Baris Kasikci, and Heiner Litz. 2022. APT-GET: profile-guided timely software prefetching. In *Proceedings of the Seventeenth European Conference on Computer Systems* (Rennes, France) *(EuroSys '22)*. Association for Computing Machinery, New York, NY, USA, 747–764. doi:10.1145/3492321.3519583

[52] Supreet Jeloka, Naveen Bharathwaj Akesh, Dennis Sylvester, and David Blaauw. 2016. A 28 nm Configurable Memory (TCAM/BCAM/SRAM) Using Push-Rule 6T Bit Cell Enabling Logic-in-Memory. *IEEE Journal of Solid-State Circuits* 51, 4 (2016), 1009–1021. doi:10.1109/JSSC.2016.2515510

[53] Doug Joseph and Dirk Grunwald. 1997. Prefetching using markov predictors. In *Proceedings of the 24th annual international symposium on Computer architecture*. 252–263.

[54] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, et al. 2020. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 790–803.

[55] Muneeb Khan and Erik Hagersten. 2014. Resource conscious prefetching for irregular applications in multicores. In *2014 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS XIV)*. IEEE, 34–43.

[56] Changkyu Kim, Tim Kaldewey, Victor W. Lee, Eric Sedlar, Anthony D. Nguyen, Nadathur Satish, Jatin Chhugani, Andrea Di Blas, and Pradeep Dubey. 2009. Sort vs. Hash revisited: fast join implementation on modern multi-core CPUs. *Proc. VLDB Endow.* 2, 2 (Aug. 2009), 1378–1389. doi:10.14778/1687553.1687564

[57] Vladimir Kiriansky, Yunming Zhang, and Saman Amarasinghe. 2016. Optimizing indirect memory references with milk. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation*. 299–312.

[58] Onur Kocberber, Boris Grot, Javier Picorel, Babak Falsafi, Kevin Lim, and Parthasarathy Ranganathan. 2013. Meet the walkers: accelerating index traversals for in-memory databases. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture* (Davis, California) *(MICRO-46)*. Association for Computing Machinery, New York, NY, USA, 468–479. doi:10.1145/2540708.2540748

[59] Akhilesh Kumar, Don Soltis, Irma Esmer, Adi Yoaz, and Sailesh Kottapalli. 2017. The new Intel Xeon scalable processor (formerly skylake-SP). In *IEEE Hot Chips Symposium (HCS)*.

[60] Snehasish Kumar, Arrvindh Shriraman, Vijayalakshmi Srinivasan, Dan Lin, and Jordon Phillips. 2014. SQRL: Hardware accelerator for collecting software data structures. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 475–476.

[61] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and Efficient Huge Page Management with Ingens. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 705–721. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/kwon

[62] Nagesh B Lakshminarayana and Hyesoon Kim. 2014. Spare register aware prefetching for graph algorithms on GPUs. In *2014 IEEE 20th international symposium on high performance computer architecture (HPCA)*. IEEE, 614–625.

[63] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2–14.

[64] Patrick Lavin, Jeffrey Young, Richard Vuduc, Jason Riedy, Aaron Vose, and Daniel Ernst. 2021. Evaluating Gather and Scatter Performance on CPUs and GPUs. In *Proceedings of the International Symposium on Memory Systems* (Washington, DC, USA) *(MEMSYS '20)*. Association for Computing Machinery, New York, NY, USA, 209–222. doi:10.1145/3422575.3422794

[65] Hyun Ryong Lee and Daniel Sanchez. 2024. Terminus: A Programmable Accelerator for Read and Update Operations on Sparse Data Structures. In *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-57)* (Austin, Texas, USA) *(MICRO '24)*.

[66] Ruipeng Li and Ulrike M. Yang. 2023. AMG2023. [Computer Software] https://doi.org/10.11578/dc.20230413.1. doi:10.11578/dc.20230413.1

[67] Xinyu Li, Lei Liu, Shengjie Yang, Lu Peng, and Jiefan Qiu. 2019. Thinking about A New Mechanism for Huge Page Management. In *Proceedings of the 10th ACM SIGOPS Asia-Pacific Workshop on Systems* (Hangzhou, China) *(APSys '19)*. Association for Computing Machinery, New York, NY, USA, 40–46. doi:10.1145/3343737.3343745

[68] Mikko H Lipasti, William J Schmidt, Steven R Kunkel, and Robert R Roediger. 1995. SPAID: Software prefetching in pointer-and call-intensive environments. In *Proceedings of the 28th annual international symposium on Microarchitecture*. IEEE, 231–236.

[69] MTP Liska, Koushik Chatterjee, D Issa, Doosoo Yoon, N Kaaz, A Tchekhovskoy, D Van Eijnatten, G Musoke, C Hesp, V Rohoza, et al. 2022. H-AMR: A New GPU-accelerated GRMHD Code for Exascale Computing with 3D Adaptive Mesh Refinement and Local Adaptive Time Stepping. *The Astrophysical Journal Supplement Series* 263, 2 (2022), 26.

[70] Los Alamos National Laboratory (LANL). 2024. ATS-5: The Fifth Advanced Technology System in the Advanced Simulation and Computing Program. https://mission.lanl.gov/advanced-simulation-and-computing/platforms/ats-5/. Accessed: 2024-11-15.

[71] Haocong Luo, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, , and Onur Mutlu. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator.

[72] S. Manegold, P. Boncz, and M. Kersten. 2002. Optimizing main-memory join on modern hardware. *IEEE Transactions on Knowledge and Data Engineering* 14, 4 (2002), 709–730. doi:10.1109/TKDE.2002.1019210

[73] Sally A McKee, William A Wulf, James H Aylor, Robert H Klenke, Maximo H Salinas, Sung I Hong, and Dee AB Weikle. 2000. Dynamic access ordering for streamed computations. *IEEE Trans. Comput.* 49, 11 (2000), 1255–1271.

[74] U. Meyer and P. Sanders. 2003. Δ-stepping: a parallelizable shortest path algorithm. *J. Algorithms* 49, 1 (Oct. 2003), 114–152. doi:10.1016/S0196-6774(03)00076-2

[75] Theodore Michailidis, Alex Delis, and Mema Roussopoulos. 2019. MEGA: overcoming traditional problems with OS huge page management. In *Proceedings of the 12th ACM International Conference on Systems and Storage* (Haifa, Israel) *(SYSTOR '19)*. Association for Computing Machinery, New York, NY, USA, 121–131. doi:10.1145/3319647.3325839

[76] Pierre Michaud. 2016. Best-offset hardware prefetching. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 469–480.

[77] William S Moses, Lorenzo Chelini, Ruizhe Zhao, and Oleksandr Zinenko. 2021. Polygeist: Raising C to polyhedral MLIR. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 45–59.

[78] A. Moshovos. 2005. RegionScout: exploiting coarse grain sharing in snoop-based coherence. In *32nd International Symposium on Computer Architecture (ISCA'05)*. 234–245. doi:10.1109/ISCA.2005.42

[79] Todd Mowry and Anoop Gupta. 1991. Tolerating latency through software-controlled prefetching in shared-memory multiprocessors. *Journal of parallel and Distributed Computing* 12, 2 (1991), 87–106.

[80] Todd C. Mowry, Monica S. Lam, and Anoop Gupta. 1992. Design and evaluation of a compiler algorithm for prefetching. In *Proceedings of the Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Boston, Massachusetts, USA) *(ASPLOS V)*. Association for Computing Machinery, New York, NY, USA, 62–73. doi:10.1145/143365.143488

[81] Anurag Mukkara, Nathan Beckmann, Maleen Abeydeera, Xiaosong Ma, and Daniel Sanchez. 2018. Exploiting locality in graph analytics through hardware-accelerated traversal scheduling. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–14.

[82] Anurag Mukkara, Nathan Beckmann, and Daniel Sanchez. 2019. PHI: Architectural support for synchronization-and bandwidth-efficient commutative scatter updates. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 1009–1022.

[83] Francisco Muñoz-Martínez, Raveesh Garg, Michael Pellauer, José L Abellán, Manuel E Acacio, and Tushar Krishna. 2023. Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 252–265.

[84] Onur Mutlu, Hyesoon Kim, and Y.N. Patt. 2005. Techniques for efficient processing in runahead execution engines. In *32nd International Symposium on Computer Architecture (ISCA'05)*. 370–381. doi:10.1109/ISCA.2005.49

[85] Onur Mutlu and Thomas Moscibroda. 2008. Parallelism-aware batch scheduling: Enhancing both performance and fairness of shared DRAM systems. *ACM SIGARCH Computer Architecture News* 36, 3 (2008), 63–74.

[86] O. Mutlu, J. Stark, C. Wilkerson, and Y.N. Patt. 2003. Runahead execution: an alternative to very large instruction windows for out-of-order processors. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings*. 129–140. doi:10.1109/HPCA.2003.1183532

[87] Ajeya Naithani, Sam Ainsworth, Timothy M Jones, and Lieven Eeckhout. 2021. Vector runahead. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 195–208.

[88] Ajeya Naithani, Josué Feliu, Almutaz Adileh, and Lieven Eeckhout. 2020. Precise runahead execution. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 397–410.

[89] Ajeya Naithani, Jaime Roelandts, Sam Ainsworth, Timothy M Jones, and Lieven Eeckhout. 2023. Decoupled vector runahead. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 17–31.

[90] Ajeya Naithani, Jaime Roelandts, Sam Ainsworth, Timothy M Jones, and Lieven Eeckhout. 2024. Decoupled Vector Runahead for Prefetching Nested Memory-Access Chains. *IEEE Micro* (2024).

[91] Karthik Nilakant, Valentin Dalibard, Amitabha Roy, and Eiko Yoneki. 2014. PrefEdge: SSD prefetcher for large-scale graph traversal. In *Proceedings of International Conference on Systems and Storage*. 1–12.

[92] Dimin Niu, Shuangchen Li, Yuhao Wang, Wei Han, Zhe Zhang, Yijin Guan, Tianchan Guan, Fei Sun, Fei Xue, Lide Duan, et al. 2022. 184QPS/W 64Mb/mm 2 3D logic-to-DRAM hybrid bonding with process-near-memory engine for recommendation system. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. IEEE, 1–3.

[93] Marcelo Orenes-Vera, Aninda Manocha, Jonathan Balkind, Fei Gao, Juan L Aragón, David Wentzlaff, and Margaret Martonosi. 2022. Tiny but mighty: designing and realizing scalable latency tolerance for manycore SoCs. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 817–830.

[94] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. Outerspace: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 724–736.

[95] Yunjie Pan, Omkar Bhalerao, C. Seshadhri, and Nishil Talati. 2024. Accurate and Fast Estimation of Temporal Motifs using Path Sampling. In *Proceedings of the International Conference on Data Mining (ICDM 2024)*. IEEE.

[96] Yunjie Pan, Jiecao Yu, Andrew Lukefahr, Reetuparna Das, and Scott Mahlke. 2023. BitSET: Bit-Serial Early Termination for Computation Reduction in Convolutional Neural Networks. *ACM Trans. Embed. Comput. Syst.* 22, 5s, Article 98 (Sept. 2023), 24 pages. doi:10.1145/3609093

[97] Ashish Panwar, Aravinda Prasad, and K. Gopinath. 2018. Making Huge Pages Actually Useful. *SIGPLAN Not.* 53, 2 (March 2018), 679–692. doi:10.1145/3296957.3173203

[98] Irma Esmer Papazian, Sailesh Kottapalli, Jeff Baxter, Jeff Chamberlain, Geetha Vedaraman, and Brian Morris. 2015. Ivy Bridge server: A converged design. *IEEE Micro* 35, 2 (2015), 16–25.

[99] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Brucek Khailany, Joel Emer, Stephen W Keckler, and William J Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. *ACM SIGARCH computer architecture news* 45, 2 (2017), 27–40.

[100] Leeor Peled, Shie Mannor, Uri Weiser, and Yoav Etsion. 2015. Semantic locality and context-based prefetching using reinforcement learning. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*. 285–297.

[101] Yue Peng, Bailin Deng, Juyong Zhang, Fanyu Geng, Wenjie Qin, and Ligang Liu. 2018. Anderson acceleration for geometry optimization and physics simulation. *ACM Transactions on Graphics (TOG)* 37, 4 (2018), 1–14.

[102] Scott Rixner. 2004. Memory Controller Optimizations for Web Servers. In *37th International Symposium on Microarchitecture (MICRO-37'04)*. 355–366. doi:10.1109/MICRO.2004.22

[103] Scott Rixner, William J. Dally, Ujval J. Kapasi, Peter Mattson, and John D. Owens. 2000. Memory access scheduling. In *Proceedings of the 27th Annual International Symposium on Computer Architecture* (Vancouver, British Columbia, Canada) *(ISCA '00)*. Association for Computing Machinery, New York, NY, USA, 128–138. doi:10.1145/339647.339668

[104] Jaime Roelandts, Ajeya Naithani, Sam Ainsworth, Timothy M Jones, and Lieven Eeckhout. 2024. Scalar Vector Runahead. (2024).

[105] Alexander Rucker, Matthew Vilim, Tian Zhao, Yaqi Zhang, Raghu Prabhakar, and Kunle Olukotun. 2021. Capstan: A vector RDA for sparsity. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1022–1035.

[106] Brian C. Schwedock, Piratach Yoovidhya, Jennifer Seibert, and Nathan Beckmann. 2022. täkō: a polymorphic cache hierarchy for general-purpose optimization of data movement. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 42–58. doi:10.1145/3470496.3527379

[107] Vivek Seshadri, Thomas Mullins, Amirali Boroumand, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch, and Todd C Mowry. 2015. Gather-scatter DRAM: In-DRAM address translation to improve the spatial locality of non-unit strided accesses. In *Proceedings of the 48th International Symposium on Microarchitecture*. 267–280.

[108] Jun Shao and Brian T Davis. 2007. A burst scheduling access reordering mechanism. In *2007 IEEE 13th International Symposium on High Performance Computer Architecture*. IEEE, 285–294.

[109] Kevin Sheridan, Christopher Scott, Jered Dominguez-Trujillo, Agustin Vaca Valverde, Patrick Lavin, Galen Shipman, Richard Vuduc, and Jeffrey Young. 2024. A Workflow for the Synthesis of Irregular Memory Access Microbenchmarks. In *Proceedings of the International Symposium on Memory Systems* (Washington D.C., USA).

[110] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramonian, Chris Wilkerson, Seth H Pugsley, and Zeshan Chishti. 2015. Efficiently prefetching complex address patterns. In *Proceedings of the 48th International Symposium on Microarchitecture*. 141–152.

[111] Yossi Shiloach and Uzi Vishkin. 1982. An O(logn) parallel connectivity algorithm. *Journal of Algorithms* 3, 1 (1982), 57–67. doi:10.1016/0196-6774(82)90008-6

[112] Galen M Shipman, Jered Dominguez-Trujillo, Kevin Sheridan, and Sriram Swaminarayan. 2022. Assessing the Memory Wall in Complex Codes. In *2022 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*. IEEE, 30–35.

[113] Galen M Shipman, Jason Pruet, David Daniel, Josh Dolence, Gary Grider, Brian M Haines, Aimee Hungerford, Stephen Poole, Tim Randles, Sriram Swaminarayan, et al. 2022. The future of HPC in nuclear security. *IEEE Internet Computing* 27, 1 (2022), 16–23.

[114] Marco Siracusa, Víctor Soria-Pardos, Francesco Sgherzi, Joshua Randall, Douglas J Joseph, Miquel Moretó Planas, and Adrià Armejach. 2023. A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors. In

Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. 1332–1346.

[115] James E. Smith. 1982. Decoupled access/execute computer architectures. In *Proceedings of the 9th Annual Symposium on Computer Architecture* (Austin, Texas, USA) *(ISCA '82)*. IEEE Computer Society Press, Washington, DC, USA, 112–119.

[116] Daniel Sorin, Mark Hill, and David Wood. 2011. *A primer on memory consistency and cache coherence*. Morgan & Claypool Publishers.

[117] Sriseshan Srikanth, Anirudh Jain, Thomas M Conte, Erik P Debenedictis, and Jeanine Cook. 2021. SortCache: intelligent cache management for accelerating sparse data workloads. *ACM Transactions on Architecture and Code Optimization (TACO)* 18, 4 (2021), 1–24.

[118] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180nm to 7nm. *Integration* 58 (2017), 74–81. doi:10.1016/j.vlsi.2017.02.002

[119] Nishil Talati, Kyle May, Armand Behroozi, Yichen Yang, Kuba Kaszyk, Christos Vasiladiotis, Tarunesh Verma, Lu Li, Brandon Nguyen, Jiawen Sun, John Magnus Morton, Agreen Ahmadi, Todd Austin, Michael O'Boyle, Scott Mahlke, Trevor Mudge, and Ronald Dreslinski. 2021. Prodigy: Improving the Memory Latency of Data-Indirect Irregular Workloads Using Hardware-Software Co-Design. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 654–667. doi:10.1109/HPCA51647.2021.00061

[120] Simon M. Tam, Harry Muljono, Min Huang, Sitaraman Iyer, Kalapi Royneogi, Nagmohan Satti, Rizwan Qureshi, Wei Chen, Tom Wang, Hubert Hsieh, Sujal Vora, and Eddie Wang. 2018. SkyLake-SP: A 14nm 28-Core xeon® processor. In *2018 IEEE International Solid-State Circuits Conference - (ISSCC)*. 34–36. doi:10.1109/ISSCC.2018.8310170

[121] Kai Troester and Ravi Bhargava. 2023. AMD Next Generation "Zen 4" Core and 4th Gen AMD EPYC™ 9004 Server CPU. In *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, 1–25.

[122] Matthew Vilim, Alexander Rucker, and Kunle Olukotun. 2021. Aurochs: An architecture for dataflow threads. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 402–415.

[123] Shu-Ting Wang, Hanyang Xu, Amin Mamandipoor, Rohan Mahapatra, Byung Hoon Ahn, Soroush Ghodrati, Krishnan Kailas, Mohammad Alian, and Hadi Esmaeilzadeh. 2024. Data motion acceleration: Chaining cross-domain multi accelerators. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1043–1062.

[124] Thomas F Wenisch, Michael Ferdman, Anastasia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2009. Practical off-chip meta-data for temporal memory streaming. In *2009 IEEE 15th International Symposium on High Performance Computer Architecture*. IEEE, 79–90.

[125] WikiChip. 2025. *Skylake (client) - Microarchitectures - Intel*. https://en.wikichip.org/wiki/intel/microarchitectures/skylake_(client)

[126] Hao Wu, Krishnendra Nathella, Joseph Pusdesris, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Temporal prefetching without the off-chip meta-data. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 996–1008.

[127] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. SpaceA: Sparse matrix vector multiplication on processing-in-memory accelerator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 570–583.

[128] W. Xing and A. Ghorbani. 2004. Weighted PageRank algorithm. In *Proceedings. Second Annual Conference on Communication Networks and Services Research, 2004.* 305–314. doi:10.1109/DNSR.2004.1344743

[129] Mingyu Yan, Xing Hu, Shuangchen Li, Abanti Basak, Han Li, Xin Ma, Itir Akgun, Yujing Feng, Peng Gu, Lei Deng, et al. 2019. Alleviating irregularity in graph analytics acceleration: A hardware/software co-design approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 615–628.

[130] Yifan Yang, Joel S. Emer, and Daniel Sanchez. 2021. SpZip: Architectural Support for Effective Data Compression In Irregular Applications. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 1069–1082. doi:10.1109/ISCA52012.2021.00087

[131] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, and Srinivas Devadas. 2015. IMP: indirect memory prefetcher. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) *(MICRO-48)*. Association for Computing Machinery, New York, NY, USA, 178–190. doi:10.1145/2830772.2830807

[132] Chao Zhang, Maximilian Bremer, Cy Chan, John Shalf, and Xiaochen Guo. 2022. ASA: A ccelerating S parse A ccumulation in Column-wise SpGEMM. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 4 (2022), 1–24.

[133] Guowei Zhang and Daniel Sanchez. 2019. Leveraging caches to accelerate hash tables and memoization. In *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 440–452.

[134] Shijin Zhang, Zidong Du, Lei Zhang, Huiying Lan, Shaoli Liu, Ling Li, Qi Guo, Tianshi Chen, and Yunji Chen. 2016. Cambricon-X: An accelerator for sparse neural networks. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.

A Artifact Appendix

A.1 Abstract

This appendix provides a step-by-step guide to reproduce the main results shown in Figures 9, 10, and 11. It includes instructions for cloning the GitHub repository, building the simulator and benchmarks, running simulations, processing the results, and plotting the charts. The simulation environment uses the event-driven gem5 [21] simulator for modeling the CPU core and DX100, and Ramulator2 [71] for accurately simulating DRAM access bandwidth and latency. Simulation infrastructure, benchmarks, automation scripts, expected results, and detailed instructions are available in our GitHub repository.

A.2 Artifact check-list (meta-information)

- **Program:** NAS [7], GAP [17], Hash-Join [11], UME [42], and Spatter [64] benchmarks are included in the `benchmarks` directory of the GitHub repository.
- **Compilation:** `g++-12`, `clang++-15`, `SCons 3.0+`, `Python 3.6+`.
- **Data set:** Benchmarks use synthetic datasets, except for the `XRAGE` dataset used by the Spatter benchmark, that is available online.
- **Metrics:** Execution time, DRAM bandwidth, row buffer hit rate, request buffer occupancy, instruction count, and cache Misses Per Kilo Instructions (MPKI).
- **Output:** Charts plotted using automation scripts available in the `results` directory of the GitHub repository.
- **Experiments:** All necessary steps are in the `README` file. A Docker image is provided on Docker Hub to simplify setup and evaluation.
- **How much disk space required?:** 6GB for simulation infrastructure and benchmarks, plus 20GB for results.
- **How much memory required?:** 35GB for each simulation.
- **How much time is needed to prepare workflow?:** 35 minutes.
- **How much time is needed to complete experiments:** Around 84 hours for serial execution (one simulation at a time using 35GB memory), or 24 hours when running four simulations in parallel (140GB memory).
- **Publicly available?:** Both GitHub and Zenodo.
- **Code licenses?:** MIT License.
- **Workflow automation framework used?:** Automation scripts are available on the `scripts` directory.

A.3 Description

This artifact has several dependencies that are mentioned on the `README` file. To simplify the evaluation process, we provide a Docker image with all required dependencies pre-installed. We recommend using the Docker container to minimize setup time and ensure a consistent environment. Use the following script to pull the image and create a container:

```
docker pull arkhadem95/dx100:latest
docker run -it --name dx100_container -v /path/to/data/
    dir:/data -w /home/ubuntu arkhadem95/dx100 bash
```

Note: `/path/to/data/dir` should point to your data directory, which must have at least 20GB of available disk space.

A.3.1 How to access. Clone the artifact from our GitHub repository using the following command.

```
git clone https://github.com/arkhadem/DX100.git
cd DX100
export GEM5_HOME=$(pwd)
```

A.3.2 Hardware Dependencies. This artifact requires 6GB of disk space for simulation infrastructure and benchmarks (`$GEM5_HOME`), and 20GB of disk space for storing simulation results (`/path/to/data`). The artifact includes 24 simulations (12 benchmarks × DX100 and baseline). Each simulation requires approximately 35GB of DRAM memory. Running all simulations on a single core takes about 84 hours to complete. These simulations can be executed in parallel to reduce total runtime, which will increase the resource requirements accordingly. For example, using four cores, the total runtime is reduced to around 24 hours, requiring approximately 140GB of memory.

A.3.3 Software Dependencies. If you choose to run the artifact locally instead of using Docker, please ensure the following software dependencies are installed: `g++-12`, `clang++-15`, `SCons >= 3.0`, `Python >= 3.6`, `protobuf >= 2.1`, and `Boost.`.

A.4 Installation

Building the artifact requires approximately 6GB of disk space and takes around 35 minutes to complete. Use the following script to build all components:

```
# Build Ramulator2
cd $GEM5_HOME/ext/ramulator2/ramulator2/
mkdir build; cd build;
cmake .. -DCMAKE_C_COMPILER=gcc-12 -DCMAKE_CXX_COMPILER=g
    ++-12
make -j

# Build M5ops
cd $GEM5_HOME/util/m5
scons build/x86/out/m5 -j8

#Build gem5
cd $GEM5_HOME
bash scripts/make.sh
bash scripts/make_fast.sh

# Build Benchmarks
cd $GEM5_HOME/benchmarks
bash build.sh
```

A.5 Experiment workflow

Before running the artifact, you can remove the current results.

```
rm $GEM5_HOME/results
```

Use the following script to run the simulations, parse the results, and plot the charts.

```
cd $GEM5_HOME
python3 scripts/benchmark.py -j NUM_THREADS -a all -dir /
    path/to/data/dir
```

Note the following configurations:

- `NUM_THREADS` specifies the number of parallel simulations. Each simulation requires approximately 35GB of memory, so set this value based on your system's available DRAM.
- `/path/to/data/dir` is the directory where gem5 simulation results will be saved. Ensure it has at least 20GB of free disk space. **Note:** If you are using the Docker container, set this path to `/data`.
- `all` runs the full pipeline, including simulation, result parsing, and figure plotting. Alternatively, you can specify `simulate`, `parse`, or `plot` to execute each step individually.

How Long Simulation Takes? Using a single thread, the complete end-to-end execution takes approximately 84 hours. By setting `NUM_THREADS` to 4, the execution time is reduced to around 24 hours that requires approximately 140GB of memory.

How to Ensure Each Step Runs Correctly? You can verify the simulation step by checking the logs located in the `/path/to/data/dir/results` directory. After parsing, the raw results will be available in the `results/results.csv` file. After plotting, you can find `png` charts in the `results` directory.

A.6 Evaluation and expected results

You can find the expected raw results (`results.csv`) as well as generated plots corresponding to Figures 9, 10, and 11 in the `results` directory of the GitHub repository.